

claude-windows / 6a82308f-69d6-43a8-9ea4-d8eaf497253d

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\subagents\agent-aa9661dcbb28364ef.jsonl`  
- SHA-256: `9477b67427e300c4b55cc53d98521409c866e7ca7115c6a9865c774f9f3327c8`  
- Source modified: `2026-06-16T03:13:57+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `subagents`  
- session_id: `6a82308f-69d6-43a8-9ea4-d8eaf497253d`
```

Transcript

1. user (2026-06-16T03:12:22.688Z)

Map the code that turns a shuttle trajectory into candidate rally windows ? the locus of an "over-merge" problem we just measured (the local detector lumps many rallies + dead time into a few giant windows). cwd = the sports highlight indexer repo.

PRIMARY: Find and read `trajectory\_to\_action\_windows` (in backend/pipeline/detectors/tracknet_runner.py, inherited by the WASB runner) and `parse\_trajectory\_csv`. Report with file:line anchors:

- The exact windowing algorithm: how "active" frames are determined, how velocity is computed (is it per-frame pixel displacement normalized by frame_width? per-second? fps-aware AT ALL?), how active regions are merged (dead_time_merge_gap) and filtered (min_window_duration).
- The parameters and their defaults/config keys: velocity_threshold (and where it's read ? indexing.<family>.velocity_threshold), dead_time_merge_gap, min_action_window_duration. Is there ANY max_window_duration cap or inactivity-based split? (Confirm absence.)
- The candidate-window dict shape (keys: start, end, peak_velocity, mean_velocity, active_frame_count, track_id_count, chunk_index, ...).
- Explicitly assess: WHY would this over-merge, and WHY would 59.94fps make it worse than ~30fps (velocity scaling)?

SECONDARY: Inventory the OTHER signals available to feed a better segmenter. For each, give file:line + what it computes + per-frame vs per-window + default on/off + how persisted:

- backend/pipeline/detectors/rally_gate.py (Gate-A net_crossings ? the held-shuttle filter)
- backend/pipeline/detectors/person_detector.py (person/court features)
- backend/pipeline/detectors/audio_features.py (audio onset cue)
- backend/eval/fusion_features.py or backend/pipeline/.../fusion_features.py (the ~5 fusion features)
- backend/pipeline/segmenters/fusion_hybrid.py (FusionSegmenter: the _enrich_candidate_stats / _select_for_handoff hooks, Gate-A/B gating)
- the candidate_segments DB table (what stats are persisted per candidate, schema)

Return a precise structured markdown map, anchor-rich, NO full file dumps. Goal: understand exactly how windowing works today (the over-merge mechanism + fps-sensitivity) and what signals already exist to build a better rally segmenter.

2. assistant (2026-06-16T03:12:24.152Z)

I'll systematically map the trajectory-to-windows pipeline and inventory the available signals. Let me start with the core windowing logic.

3. user (2026-06-16T03:12:24.629Z)

```
1      """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).  
2  
3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living  
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:  
5
```

```

6         1. Translate the Windows video path to its WSL-visible form (and optionally
7           stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8         2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12           pipeline (AI handoff, DB population) is unchanged.
13
14    Nothing here imports TensorFlow; all model code stays in WSL.
15    """
16
17    import os
18    import csv
19    import shlex
20    import logging
21    import subprocess
22    from dataclasses import dataclass
23    from typing import List, Dict, Any, Optional, Tuple
24
25    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27    logger = logging.getLogger(__name__)
28
29
30    @dataclass
31    class TrackNetConfig:
32        """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33        distro: str = "Ubuntu"
34        repo_dir: str = "~/models/TrackNetV4" # WSL path to the cloned repo
35        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36        conda_env: str = "TrackNetV4"
37        weights_path: str = "" # WSL path to the .h5/.keras weights
38        (REQUIRED)
39        queue_length: int = 5
40        stage_in_wsl: bool = True # copy video into WSL fs first
41        (perf)
42        wsl_stage_dir: str = "~/clips" # where staged videos/outputs live
43        in WSL
44        timeout_sec: int = 1800 # 30 min; kills a hung call (0 = no
timeout)
45        keep_frames: bool = False # retain staged video after success
46        (debug only)
47        min_free_gb: float = 0.0 # warn if WSL free space below this
(0 = off)
48
49
50    @classmethod
51    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
52        tn = (idx_cfg or {}).get("tracknet", {}) or {}
53        return cls(
54            distro=tn.get("wsl_distro", "Ubuntu"),
55            repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
56            conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
57            conda_env=tn.get("conda_env", "TrackNetV4"),
58            weights_path=tn.get("weights_path", ""),
59            queue_length=int(tn.get("queue_length", 5)),
60            stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
61            wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
62            timeout_sec=int(tn.get("timeout_sec", 1800)),
63            keep_frames=bool(tn.get("keep_frames", False)),
64            min_free_gb=float(tn.get("min_free_gb", 0.0)),
65        )

```

```

63 def to_wsl_mnt_path(win_path: str) -> str:
64     """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66     Already-POSIX paths (starting with / or ~) are returned unchanged so the
67     same helper is safe to call on either kind of path.
68     """
69     p = str(win_path)
70     if p.startswith("/") or p.startswith("~"):
71         return p
72     p = p.replace("\\", "/")
73     if len(p) >= 2 and p[1] == ":":
74         drive = p[0].lower()
75         return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU')))\n"
107         )
108         try:
109             res = self._wsl_bash(cmd)
110             except FileNotFoundError:
111                 return False, "`wsl` not found ? is this running on Windows with WSL2
112                 installed?"
113             except subprocess.TimeoutExpired:
114                 return False, "WSL healthcheck timed out."
115             out = (res.stdout or "").strip()
116             if res.returncode != 0:
117                 return False, f"WSL env check failed: {(res.stderr or out).strip()}"
118             return True, out
119
120     # ----- inference
121
122     def run_predict(self, video_win_path: str, output_win_dir: str,
123                    video_id: Optional[str] = None) -> Optional[str]:
124         """Run predict.py on a video. Returns the Windows path to the produced CSV, or
125         None.
126
127         ``output_win_dir`` is a Windows directory (under the repo's output/) that

```

```

125 WSL writes into via its /mnt view, so the Windows process can read the CSV
126 back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
127 therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
128 a filename cannot collide on the output CSV.
129 """
130 if not self.cfg.weights_path:
131     logger.error(
132         "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
133         "in config.json to the WSL path of your trained TrackNetV4 weights."
134     )
135     return None
136
137 # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
138 # relative paths through unchanged, so WSL would resolve them against the
139 # predict.py cwd instead of the repo (same bug class as wasb_runner).
140 output_win_dir = os.path.abspath(output_win_dir)
141 os.makedirs(output_win_dir, exist_ok=True)
142 # Normalize separators so basename/splitext work when tests (or collector)
143 # pass Windows-style paths on a Linux host.
144 video_win_path = str(video_win_path).replace("\\", "/")
145 base = os.path.basename(video_win_path)
146 stem, ext = os.path.splitext(base)
147
148 # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149 if ext.lower() not in (".mp4", ".avi"):
150     logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151     return None
152
153 wsl_out = to_wsl_mnt_path(output_win_dir)
154 # Namespace by video_id so two same-filename videos don't collide on the CSV.
155 # predict.py derives its output name from the (staged) video stem, so staging
156 # under the namespaced name propagates into "<key>_predict.csv".
157 key = f"{stem}__{video_id[:12]}" if video_id else stem
158 expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160 # Resolve the video path WSL will read.
161 if self.cfg.stage_in_wsl:
162     staged = self._stage_video(video_win_path, f"{key}{ext}")
163     if staged is None:
164         return None
165     wsl_video = staged
166 else:
167     # No staging: predict.py reads /mnt directly and writes
168     # "<stem>_predict.csv".
169     # We cannot rename its output, so fall back to the un-namespaced name.
170     wsl_video = to_wsl_mnt_path(video_win_path)
171     expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
172
173 if self.cfg.min_free_gb > 0:
174     free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
175     if free is not None and free < self.cfg.min_free_gb:
176         logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
177             "consider running tools/cleanup_caches.py.",
178             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
179
180 cmd = (
181     f"conda activate {self.cfg.conda_env} && "
182     f"cd {self.cfg.repo_dir}/src && "
183     f"python predict.py "
184     f"--video_path {wsl_tilde_quote(wsl_video)} "
185     f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
186     f"--output_dir {wsl_tilde_quote(wsl_out)} "
187     f"--queue_length {self.cfg.queue_length}"
188 )
189 logger.info("Running TrackNet inference on %s ...", base)

```

```

189         try:
190             res = self._wsl_bash(cmd)
191         except subprocess.TimeoutExpired:
192             logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
193             return None
194
195         if res.returncode != 0:
196             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
197             return None
198
199         if not os.path.exists(expected_csv_win):
200             logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
201             return None
202         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205         # regenerable intermediate). On earlier failure we return above without
cleaning.
206         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207             logger.info("Cache hygiene: removing staged video for %s "
208                         "(set indexing.tracknet.keep_frames=true to retain).", key)
209             self._wsl_rm_rf([wsl_video])
210         return expected_csv_win
211
212     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         reason = self.wsl_source_unreadable_reason(src_mnt)
219         if reason:
220             logger.error("Cannot stage video into WSL: %s", reason)
221             return None
222         dest = f"{self.cfg.wsl_stage_dir}/{base}"
223         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
224         try:
225             res = self._wsl_bash(cmd)
226         except subprocess.TimeoutExpired:
227             logger.error("Staging video into WSL timed out.")
228             return None
229         if res.returncode != 0 or "OK" not in (res.stdout or ""):
230             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
231             return None
232         return dest
233
234     # ----- CSV -> windows
235
236     @staticmethod
237     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239         points: List[TrajectoryPoint] = []
240         with open(csv_win_path, newline="") as f:
241             reader = csv.DictReader(f)
242             for row in reader:
243                 try:
244                     points.append(TrajectoryPoint(
245                         frame=int(row["Frame"]),
246                         visible=int(row["Visibility"]) == 1,
247                         x=float(row["X"]),

```

```

248             y=float(row["Y"]),
249         ))
250     except (KeyError, ValueError):
251         continue
252     points.sort(key=lambda p: p.frame)
253     return points
254
255 @staticmethod
256 def trajectory_to_action_windows(
257     points: List[TrajectoryPoint],
258     fps: float,
259     frame_width: float,
260     chunk_start: float = 0.0,
261     velocity_thresh: float = 0.02,
262     merge_gap: float = 2.0,
263     min_window_duration: float = 2.0,
264     chunk_index: Optional[int] = None,
265 ) -> List[Dict[str, Any]]:
266     """Convert a shuttle trajectory into candidate rally windows.
267
268     A frame is "active" when the shuttle is visible AND its inter-frame
269     displacement (normalised by frame width, per second) exceeds
270     ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
271     ``merge_gap`` seconds of each other are grouped into one window; short
272     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
273     shape feeding the AI-handoff phase is identical.
274     """
275     if fps <= 0:
276         fps = 30.0
277     if frame_width <= 0:
278         frame_width = 1280.0
279
280     # Compute per-frame normalised velocity from the last *visible* point.
281     active = [] # (frame_idx, velocity)
282     prev: Optional[TrajectoryPoint] = None
283     for p in points:
284         if not p.visible:
285             prev = None # break the trajectory; absent shuttle = not in flight
286             continue
287         if prev is not None:
288             dframes = p.frame - prev.frame
289             if dframes > 0:
290                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
291                 velocity = (dist / frame_width) * (fps / dframes)
292                 if velocity > velocity_thresh:
293                     active.append((p.frame, velocity))
294             prev = p
295
296     if not active:
297         return []
298
299     max_gap_frames = int(merge_gap * fps)
300
301     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
302         vels = [v for _, v in group]
303         return {
304             "start": group[0][0] / fps + chunk_start,
305             "end": group[-1][0] / fps + chunk_start,
306             "active_frame_count": len(group),
307             "peak_velocity": max(vels),
308             "mean_velocity": sum(vels) / len(vels),
309             "track_id_count": 1, # single shuttle; kept for window-shape parity
310             "chunk_index": chunk_index,
311         }
312

```

```

313         windows = []
314         group = [active[0]]
315         for af in active[1:]:
316             if af[0] - group[-1][0] <= max_gap_frames:
317                 group.append(af)
318             else:
319                 windows.append(_finalize(group))
320                 group = [af]
321         windows.append(_finalize(group))
322
323         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
324

```

4. assistant (2026-06-16T03:12:27.012Z)

Good start. Now let me search for where `trajectory_to_action_windows` is actually called (parameters source) and look at the other detector signals.

5. user (2026-06-16T03:12:27.507Z)

Found 16 files

```

backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\detectors\base.py
backend\pipeline\detectors\tracknet_runner.py
backend\eval>windowing.py
backend\eval\serve_contrast.py
backend\eval\distill_local.py
backend\eval\served_gate_a.py
backend\eval\fusion_golden.py
backend\pipeline\detectors\rally_gate.py
backend\eval\calibrate_local.py
backend\eval\rally_seq_proto.py
backend\eval\eval_partial_labels.py
backend\eval\calibrate_wasb.py
backend\eval\export_wasb_segments.py

```

6. user (2026-06-16T03:12:27.643Z)

```

1      """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3      WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4      the opponent for service after losing a point, that single trajectory looks like a
5      rally. The discriminator is structural, not duration-based: a real rally has the
6      shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7      single service feed crosses ~once.
8
9      This module is camera-agnostic and needs NO new model ? it works on the trajectory
10     we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11     axis with the larger shuttle spread) and the net position (median shuttle coord on
12     that axis ? the shuttle clusters at the two ends and crosses the net region often,
13     so the median sits near the net), then counts sign changes of (coord - net) across
14     the visible points inside a candidate window, with a deadband to ignore jitter.
15
16     Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17     Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18     windows from trajectory_to_action_windows, or to fold into it later.
19     """
20     from __future__ import annotations
21
22     from dataclasses import dataclass
23     from typing import List, Optional, Sequence, Tuple
24

```

```

25     Interval = Tuple[float, float]
26
27
28 @dataclass
29 class GatedWindow:
30     start: float
31     end: float
32     crossings: int
33     visible_points: int
34     kept: bool
35
36
37 def _spread(vals: Sequence[float]) -> float:
38     """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39     if len(vals) < 2:
40         return 0.0
41     s = sorted(vals)
42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89

```



```

90
91     def gate_windows(points, windows: List[Interval], fps: float,
92                       min_crossings: int = 2, deadband_frac: float = 0.06,
93                       estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94         """Annotate each (start,end) window with its shuttle net-crossing count and a
95         keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96         axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98         `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99         `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
100         per window ? same result, avoids the per-window re-estimation."""
101         axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102         deadband = deadband_frac * span
103         out: List[GatedWindow] = []
104         for (s, e) in windows:
105             wp = _points_in(points, fps, s, e)
106             c = count_net_crossings(wp, axis, net, deadband)
107             vis = sum(1 for p in wp if p.visible)
108             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                                   kept=c >= min_crossings))
110         return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

7. user (2026-06-16T03:12:28.062Z)

```

1     """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3     Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4     the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5     `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6     next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7     later the learned classifier reads its persisted features. Keeping detection here (not
in
8     the segmenter) is what makes "add a cue = register a producer" true.
9
10    Behaviour is unchanged from the in-lined version ? only the home moved:
11    - `lazy_load_model`           ? load a permissively-licensed torchvision detector
12    - `make_tracker`             ? a fresh supervision ByteTrack per chunk
13    - `detect_and_track`         ? one frame ? confident persons with persistent IDs
14    - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15      with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16      that feed the candidate `stats` JSON and the learned fusion features.
17
18    The model libs (torchvision, supervision) are imported lazily inside the methods, and
the
19    detector weights load only on first detection (`lazy_load_model`) ? so importing the
20    segmenter registry never loads a detector model, and tests can mock the seams. (torch +
cv2
21    are top-level imports here; CI's import-smoke stubs them, and they're present on the GPU
22    machine.) Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
23    (ADR-005; ultralytics' AGPL YOLO was dropped).
24    """
25    import logging
26    from typing import Any, Callable, Dict, List, Optional, Tuple
27

```

```

28     import cv2
29     import torch
30
31     from backend.utils.geometry import get_velocity_normalised
32
33     logger = logging.getLogger(__name__)
34
35     # torchvision detection models are trained on COCO where the "person" category is
36     # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
37     # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
38     _PERSON_CLASS_ID = 1
39
40     # Permissively-licensed torchvision detectors selectable via config
41     # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
42     _DETECTOR_FACTORIES = {
43         "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
44         "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
45         "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
46     }
47     _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
48
49
50     class PersonDetector:
51         """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
52
53         Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
54         ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
55         min_action_window_duration}``); the model loads lazily on first use.
56         """
57
58         def __init__(self, config: Dict[str, Any]):
59             self.config = config
60             # Dynamic torchvision module (None until lazy_load_model); typed Any so the
61             # `self.model([tensor])` inference call type-checks without a quarantine.
62             self.model: Any = None
63             self.device: Optional[str] = None
64
65         def lazy_load_model(self) -> None:
66             if self.model is not None:
67                 return
68
69             # Imported lazily so the module imports without torchvision present and so
70             # the test suite (which pre-sets self.model) never pulls in model weights.
71             from torchvision.models import detection as tv_detection
72
73             idx_cfg = self.config.get("indexing", {})
74             use_gpu = idx_cfg.get("use_gpu", True)
75             self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
76             if use_gpu and not torch.cuda.is_available():
77                 logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
78
79             model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
80             if model_name not in _DETECTOR_FACTORIES:
81                 logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
82                 model_name = _DEFAULT_DETECTOR
83
84             builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
85             logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
86             # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
87             self.model = builder(weights="DEFAULT")
88             self.model.eval()
89             self.model.to(self.device)
90

```

```

91     def make_tracker(self, fps: float) -> Any:
92         """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
93
94         Imported lazily so tests can mock this method without supervision installed.
95         """
96         import supervision as sv
97         frame_rate = max(1, int(round(fps)))
98         return sv.ByteTrack(frame_rate=frame_rate)
99
100     def detect_and_track(self, frame: Any, tracker: Any,

```

8. user (2026-06-16T03:12:28.274Z)

```

1     """Pure per-window audio-cue features for rally fusion (fusion P4).
2
3     NO I/O ? the audio hit detection lives in `backend/pipeline/audio/hit_detector.py`
4     (`detect_hits_in_video` -> `List[HitEvent]`); this only turns those hit events into a
few
5     per-window features the learned scorer can weight. Mirrors `fusion_features.py`
discipline.
6
7     Honest expectation (see the `audio-e1-findings` memory): the classical-onset audio cue
had
8     only ~2.2x discrimination and was NOT adopted standalone ? so this is "add the signal as
a
9     column and let the golden-set harness decide", default-OFF, not "adopt audio".
10    """
11    from __future__ import annotations
12
13    from typing import Dict, List
14
15    AUDIO_FEATURE_NAMES = ["audio_hit_count", "audio_hit_rate", "audio_hit_strength_mean"]
16
17
18    def compute_audio_features(hits: List, window_start: float, window_end: float) ->
Dict[str, float]:
19        """Audio features for one window from a video's HitEvents (each has ``.t``,
``.strength``).
20
21        A real rally is a sustained volley of racket/shuttle contacts; dead time / a single
22        service feed has few. Returns hit count, hits/second, and mean strength ? all 0.0 on
an
23        empty window. ``[start, end)`` half-open so a hit on a boundary belongs to one
window only.
24        """
25        dur = max(1e-6, window_end - window_start)
26        in_win = [h for h in (hits or []) if window_start <= h.t < window_end]
27        if not in_win:
28            return {k: 0.0 for k in AUDIO_FEATURE_NAMES}
29        n = len(in_win)
30        return {
31            "audio_hit_count": float(n),
32            "audio_hit_rate": n / dur,
33            "audio_hit_strength_mean": sum(h.strength for h in in_win) / n,
34        }
35

```

9. assistant (2026-06-16T03:12:29.920Z)

Now let me find the fusion_features and FusionSegmenter files:

10. user (2026-06-16T03:12:30.245Z)

backend\pipeline\detectors\fusion_features.py

```
.claude\worktrees\bhi-execpolicy\backend\pipeline\detectors\fusion_features.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\fusion_features.py
.claude\worktrees\local-noai\backend\pipeline\detectors\fusion_features.py
```

11. user (2026-06-16T03:12:30.449Z)

```
.claude\worktrees\bhi-execpolicy\backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\local-noai\backend\pipeline\segmenters\fusion_hybrid.py
```

12. user (2026-06-16T03:12:30.991Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\fusion_audio.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\fusion_compare.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\fusion_golden.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\fusion_features.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\base.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\fusion_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\motion.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\tracknet_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\trajectory_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\wasb_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\yolo_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py
```

13. assistant (2026-06-16T03:12:32.359Z)

Now let me read the key files:

14. user (2026-06-16T03:12:32.799Z)

```
1      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3      NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4      math is unit-testable in a GPU-free dev container. The detection (person boxes,
5      shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6      module only turns those signals into a handful of scale/translation-invariant
7      features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8      memorise this court/camera and fail to generalise off a small labelled set).
9
10     Conventions:
11     - A person box is `(x1, y1, x2, y2)` in pixels; `(track_id, box)` when motion
needs it.
12     - per_frame is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13     index (so it aligns directly with the shuttle trajectory's .frame ? no clip re-
timing).
14     - A shuttle point has .frame, .visible, .x, .y (pixels) ? the WASB
trajectory shape.
15     - court_polygon is normalised 0-1 [(x, y), ...] or None (None ? no mask,
every
16     detection counts ? the player-count-only mode).
17     - net is (axis 'x'|'y', position 0-1 normalised) or None (None ? two-sided =
0.0).
18
19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except mean_player_motion (a non-negative normalised speed).
21     """
```

```

22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31         """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32         (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33         if polygon is None or len(polygon) < 3:
34             return False
35         x, y = point
36         inside = False
37         n = len(polygon)
38         j = n - 1
39         for i in range(n):
40             xi, yi = polygon[i]
41             xj, yj = polygon[j]
42             # Does the horizontal ray at y cross edge (i, j)?
43             if (yi > y) != (yj > y):
44                 x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                 if x < x_cross:
46                     inside = not inside
47             j = i
48         return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71         point meaningless, so a masked check returns False rather than testing a bogus
72         (0,0)."""
73         if court_polygon is None:
74             return True
75         if frame_width <= 0 or frame_height <= 0:
76             return False
77         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
78
79     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
80                       frame_height: float,
81                       court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:

```

```

82         """Per-frame count of in-court persons (one entry per sampled frame)."""
83         return [
84             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
85 court_polygon))
86             for _frame_idx, dets in sorted(per_frame.items())
87         ]
88
89     def _median(vals: Sequence[float]) -> float:
90         if not vals:
91             return 0.0
92         s = sorted(vals)
93         n = len(s)
94         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97     def players_in_court(counts: Sequence[int]) -> float:
98         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99         return _median(list(counts))
100
101
102     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103         """Fraction of sampled frames with >= `min_players` in court (track stability vs
104 flicker)."""
105         if not counts:
106             return 0.0
107         return sum(1 for c in counts if c >= min_players) / len(counts)
108
109     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110 frame_width: float, frame_height: float) -> float:
111         """Mean per-track centre displacement between consecutive sampled frames, with x
112 normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113 not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114         if frame_width <= 0 or len(per_frame) < 2:
115             return 0.0
116         frames = sorted(per_frame.keys())
117         steps: List[float] = []
118         for a, b in zip(frames, frames[1:]):
119             prev = {tid: box for tid, box in per_frame[a]}
120             for tid, box in per_frame[b]:
121                 if tid in prev:
122                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125         if not steps:
126             return 0.0
127         return sum(steps) / len(steps)
128
129
130     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
131 frame_height: float, net: Optional[Net],
132 court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
137         0.0.
138         """
139         if net is None or not per_frame:
140             return 0.0
141         axis, pos = net
142         both = 0
143         for _frame_idx, dets in per_frame.items():
144             near, far = False, False

```



```

204                 min_players: int = 2) -> Dict[str, float]:
205         """Aggregate the per-window person features from per-frame detections + the in-
window
206         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
207         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
208         return {
209             "players_in_court": players_in_court(counts),
210             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
211             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212             "in_court_ratio": in_court_ratio(counts, min_players),
213             "shuttle_player_colocation": shuttle_player_colocation(
214                 per_frame, shuttle_points, frame_width, frame_height),
215         }
216

```

15. user (2026-06-16T03:12:32.979Z)

```

1         """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3         Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4         TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5         do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6         shuttle-seeded window via two overridable seams the engine exposes:
7
8         - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16        - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19        **Persisted candidates remain the full superset** regardless, so the offline
experiment
20        harness can re-score any variant.
21
22        Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23        nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24        and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25        segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26        experiment harness (EXPERIMENT_HARNESS.md), never silently.
27        """
28        from typing import Any, Dict, List, Optional, Tuple
29
30        from backend.pipeline.segmenters.base import segmenter_registry
31        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32        from backend.pipeline.detectors.base import DetectorRunner
33        from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34        from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35        from backend.pipeline.detectors import rally_gate
36        from backend.pipeline.detectors import fusion_features as ff
37

```



```

38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
41 B)."""
42         family = "fusion"
43         display_label = "Fusion"
44
45         def __init__(self, db: Any, config: Any):
46             super().__init__(db, config)
47             # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48             self._person: Optional[Any] = None
49             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50             # over the whole trajectory once per candidate window (it depends only on
`points`).
51             self._axis_cache_key: Optional[int] = None
52             self._axis_cache: Optional[tuple] = None
53
54         @property
55         def name(self) -> str:
56             return "fusion_hybrid"
57
58         @property
59         def description(self) -> str:
60             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                     "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                     "AI provider sets semantic boundaries.")
63
64         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65             substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66             if substrate == "tracknet":
67                 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                 return TrackNetRunner(cfg), {
69                     "weights_path": cfg.weights_path,
70                     "queue_length": cfg.queue_length,
71                     "fusion_substrate": "tracknet",
72                 }
73             wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74             return WasbRunner(wcfg, {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"})
75
76         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                     frame_width: float, video_path: str,
78                                     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79             stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80             # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81             # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82             # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83             gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                                  min_crossings=0,
estimate=self._axis_estimate(points))
85             stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87             fcfg = idx_cfg.get("fusion", {})
88             if fcfg.get("cue_flags", {}).get("person", False):
89                 stats.update(self._person_features(cw, points, fps, frame_width, video_path,

```

```

fcfg))
90         return stats
91
92         # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
93         # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
94         # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
95         # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
96         # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
97         # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
98         # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
99
100        def _axis_estimate(self, points: List[Any]) -> tuple:
101            """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
102            computed once per video, not once per candidate window (it depends only on
points)."""
103            key = id(points)
104            if self._axis_cache_key != key or self._axis_cache is None:
105                self._axis_cache_key = key
106                self._axis_cache = rally_gate.estimate_play_axis(points)
107            est = self._axis_cache
108            assert est is not None
109            return est
110
111        def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
112                             frame_width: float, video_path: str,
113                             fcfg: Dict[str, Any]) -> Dict[str, float]:
114            """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
115            frame range and compute the scale/translation-invariant person features. Lazily
builds
116            ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
117            # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
118            from backend.pipeline.detectors.person_detector import PersonDetector
119            if self._person is None:
120                self._person = PersonDetector(self.config)
121
122            frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
123            # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
124            # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
125            start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
126            det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
127            fw = det["frame_width"] or frame_width
128            fh = det["frame_height"]
129            # If we can't trust the frame dims, every normalised feature would silently
collapse
130            # to -0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
131            if fw <= 0 or fh <= 0:
132                return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
133
134            poly = fcfg.get("court_polygon")
135            polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
136            net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
137            shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
138            return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,

```

```

139             court_polygon=polygon, net=net)
140
141     def _select_for_handoff(self, windows: List[Dict[str, Any]],
142                             window_stats: List[Dict[str, Any]],
143                             idx_cfg: Dict[str, Any]) -> List[int]:
144         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
145         (0) keep every window ? identical to the substrate. Persisted candidates are
146         untouched (full superset)."""
147         fcfg = idx_cfg.get("fusion", {})
148         min_crossings = int(fcfg.get("min_crossings", 0))
149         min_players = int(fcfg.get("min_players", 0))
150         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
151
152         keep: List[int] = []
153         for i, st in enumerate(window_stats):
154             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
155                 continue
156             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
157                 continue
158             keep.append(i)
159         return keep
160
161
162     segmenter_registry.register(FusionSegmenter)
163

```

16. assistant (2026-06-16T03:12:38.229Z)

Now let me check where trajectory_to_action_windows is invoked and what parameters are used. Let me also get the DB schema:

17. user (2026-06-16T03:12:39.181Z)

```

backend/pipeline/detectors/base.py:33:   TrackNetRunner.parse_trajectory_csv and
.trajectory_to_action_windows (they are
backend/pipeline/detectors/base.py:50:- Write the exact same CSV format so
trajectory_to_action_windows works verbatim.
backend/pipeline/detectors/base.py:191:   trajectory_to_action_windows: Callable[..., Any]
backend/pipeline/detectors/rally_gate.py:18:windows from trajectory_to_action_windows, or to
fold into it later.
backend/pipeline/detectors/tracknet_runner.py:256:   def trajectory_to_action_windows(
backend/pipeline/detectors/wasb_runner.py:20:# parse_trajectory_csv /
trajectory_to_action_windows are @staticmethods on TrackNetRunner.
backend/pipeline/detectors/wasb_runner.py:199:   trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
backend/pipeline/segmenters/trajectory_hybrid.py:197:       all_candidate_windows =
runner.trajectory_to_action_windows(

```

18. user (2026-06-16T03:12:39.973Z)

```

backend\config\models.py:43:   velocity_threshold: float = 0.02
backend\config\models.py:69:   velocity_threshold: float = 0.02
backend\config\models.py:97:   # indexing.<family>.velocity_threshold). Default matches the
substrates' 0.02; set
backend\config\models.py:98:   # equal to indexing.wasb.velocity_threshold to A/B fusion
against a tuned wasb run.
backend\config\models.py:99:   velocity_threshold: float = 0.02
backend\config\models.py:122:   dead_time_merge_gap: float = 2.0
backend\config\models.py:127:   yolo_velocity_threshold: float = 0.15
backend\config\models.py:131:   min_action_window_duration: float = 2.0
backend\eval\calibrate_local.py:6:   WASB trajectory CSV ->
trajectory_to_action_windows(velocity, merge_gap, min_dur)

```

```

backend\eval\calibrate_local.py:34:# (velocity_thresh, merge_gap, min_window_duration,
min_crossings)
backend\eval\calibrate_local.py:48:         velocity_thresh=vt, merge_gap=mg,
min_window_duration=mwd,
backend\pipeline\detectors\person_detector.py:55:     min_action_window_duration}``); the model
loads lazily on first use.
backend\pipeline\detectors\person_detector.py:141:
velocity_thresh: float, merge_gap: float,
backend\pipeline\detectors\person_detector.py:206:         if velocity >
velocity_thresh:
backend\pipeline\detectors\person_detector.py:227:         max_gap_frames = int(merge_gap * fps)
backend\pipeline\detectors\person_detector.py:253:         min_window_duration =
self.config.get("indexing", {}).get("min_action_window_duration", 2.0)
backend\pipeline\detectors\person_detector.py:254:         windows = [w for w in windows if
(w["end"] - w["start"]) >= min_window_duration]
backend\eval\calibrate_wasb.py:28:Config = Tuple[float, float, float] # (velocity_thresh,
merge_gap, min_window_duration)
backend\eval\calibrate_wasb.py:52:         velocity_thresh=vt, merge_gap=mg,
min_window_duration=mwd,
backend\eval\distill_local.py:43:# candidates (the classifier removes the junk).
(velocity_thresh, merge_gap, min_dur).
backend\eval\distill_local.py:59:         velocity_thresh=vt, merge_gap=mg,
min_window_duration=mwd)
backend\eval\distill_local.py:81:         velocity_thresh=vt, merge_gap=mg,
min_window_duration=mwd)
backend\pipeline\segmenters\motion.py:44:         dead_time_merge_gap =
idx_cfg.get("dead_time_merge_gap", 2.0)
backend\pipeline\segmenters\motion.py:141:         # Post-Processing: 1. Merge segments separated
by less than dead_time_merge_gap
backend\pipeline\segmenters\motion.py:146:         if next_start - curr_end <
dead_time_merge_gap:
backend\pipeline\detectors\tracknet_runner.py:261:         velocity_thresh: float = 0.02,
backend\pipeline\detectors\tracknet_runner.py:262:         merge_gap: float = 2.0,
backend\pipeline\detectors\tracknet_runner.py:263:         min_window_duration: float = 2.0,
backend\pipeline\detectors\tracknet_runner.py:270:         ``velocity_thresh`` ? i.e. the shuttle
is in flight. Active frames within
backend\pipeline\detectors\tracknet_runner.py:271:         ``merge_gap`` seconds of each other
are grouped into one window; short
backend\pipeline\detectors\tracknet_runner.py:292:         if velocity >
velocity_thresh:
backend\pipeline\detectors\tracknet_runner.py:299:         max_gap_frames = int(merge_gap * fps)
backend\pipeline\detectors\tracknet_runner.py:323:         return [w for w in windows if
(w["end"] - w["start"]) >= min_window_duration]
backend\eval\eval_partial_labels.py:132:# min_window_duration DOWN (recover short 3-5s rallies)
and merge_gap UP (heal
backend\eval\eval_partial_labels.py:146:     """Grid-search (velocity, merge_gap, min_dur) x gate
vs human labels, in-window.
backend\eval\eval_partial_labels.py:186:     print(f"\nbest: velocity={b['cfg'][0]}
merge_gap={b['cfg'][1]} min_dur={b['cfg'][2]} ")
backend\pipeline\segmenters\trajectory_hybrid.py:47:     #: config section under `indexing`
(velocity_threshold lives there), the
backend\pipeline\segmenters\trajectory_hybrid.py:160:         velocity_thresh =
idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
backend\pipeline\segmenters\trajectory_hybrid.py:161:         merge_gap =
idx_cfg.get("dead_time_merge_gap", 2.0)
backend\pipeline\segmenters\trajectory_hybrid.py:162:         min_window_duration =
idx_cfg.get("min_action_window_duration", 2.0)
backend\pipeline\segmenters\trajectory_hybrid.py:170:         "velocity_thresh":
velocity_thresh,
backend\pipeline\segmenters\trajectory_hybrid.py:171:         "merge_gap": merge_gap,
backend\pipeline\segmenters\trajectory_hybrid.py:172:         "min_action_window_duration":
min_window_duration,
backend\pipeline\segmenters\trajectory_hybrid.py:199:
velocity_thresh=velocity_thresh, merge_gap=merge_gap,
backend\pipeline\segmenters\trajectory_hybrid.py:200:

```

```

min_window_duration=min_window_duration, chunk_index=0,
backend\eval\export_wasb_segments.py:35:# The tuned config from the first per-video calibration
(velocity, merge_gap, min_dur);
backend\eval\export_wasb_segments.py:219:    cfg = (args.velocity_thresh, args.merge_gap,
args.min_duration)
backend\eval\experiment.py:103:        - ``threshold`` ? classifier probability cutoff.
``merge_gap`` ? post-filter
backend\eval\experiment.py:116:    merge_gap: float = 1.0
backend\eval\experiment.py:138:        "merge_gap": self.merge_gap,

```

[Showing results with pagination = limit: 50]

19. assistant (2026-06-16T03:12:41.581Z)

Good! Now let me check the trajectory_hybrid segmenter to see the full call context:

20. user (2026-06-16T03:12:42.220Z)

```

1      """Shared engine for trajectory-substrate hybrid segmenters.
2
3      tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4      standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5      now the single home for the whole pipeline shape:
6
7          healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8          -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10     A concrete subclass supplies only its identity and runner wiring:
11     - ``family``          ? config section under ``indexing.*``, output subdir, and
12                           the ``<family>-hybrid-<model>`` detector tag in metadata.
13     - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14     - ``name`` / ``description`` properties (the registry contract).
15     - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                                     detection_params extras such as weights_path).
17
18     This base is deliberately NOT registered; only concrete subclasses register.
19     """
20     import os
21     import time
22     import logging
23     import concurrent.futures
24     from typing import Any, Dict, List, Optional, Tuple
25
26     import cv2
27
28     from backend.pipeline.segmenters.base import BasePlaySegmenter
29     from backend.utils.video import get_video_duration, extract_video_chunk
30     from backend.sports import sport_registry
31     from backend.providers import provider_registry
32     from backend.pipeline.detectors.base import DetectorRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46         boundaries."""

```

```

47     #: config section under `indexing` (velocity_threshold lives there), the
48     #: output subdir for trajectories, and the metadata detector-tag prefix.
49     family: str = ""
50     #: human-readable label used in log lines.
51     display_label: str = ""
52
53     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
54         """Return (runner, detector-specific detection_params extras)."""
55         raise NotImplementedError
56
57     @staticmethod
58     def _probe_fps_width(video_path: str) -> tuple:
59         """Return (fps, frame_width) for a video, with sensible fallbacks."""
60         cap = cv2.VideoCapture(video_path)
61         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
62         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
63         cap.release()
64         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
65
66     @staticmethod
67     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
68         """Provider-reported exchange count, else a duration-based estimate.
69
70         One canonical implementation ? the twins had drifted (wasb relied on
71         ``int(None)`` raising into the fallback; same result, by accident).
72         """
73         shot_count = segment.get("shuttle_exchange_count")
74         if shot_count is None:
75             return max(3, int(duration / 0.8))
76         try:
77             return int(shot_count)
78         except (ValueError, TypeError):
79             return max(3, int(duration / 0.8))
80
81     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
82     -----
83     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
84                                frame_width: float, video_path: str,
85                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
86         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
87         BASE
88         identical
89         returns exactly the 4 motion keys, so the trajectory twins persist byte-
90         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
91         features). ``points`` are the whole-video trajectory points
92         (TrajectoryPoint)."""
93         return {
94             "peak_velocity": cw["peak_velocity"],
95             "mean_velocity": cw["mean_velocity"],
96             "active_frame_count": cw["active_frame_count"],
97             "track_id_count": cw["track_id_count"],
98         }
99
100     def _select_for_handoff(self, windows: List[Dict[str, Any]],
101                            window_stats: List[Dict[str, Any]],
102                            idx_cfg: Dict[str, Any]) -> List[int]:
103         """Indices of the candidate windows that proceed to the AI handoff (and thus may
104         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
105         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
106         Persisted candidates are NOT affected ? they remain the full superset for
107         offline
108         re-scoring."""
109         return list(range(len(windows)))
110

```

```

106     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                       player_pool: Optional[List[str]] = None,
108                       max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109         # override-ignore: the ABC declares the Result contract (Success|Failure)
110         # but every live segmenter still returns a bare list ? the documented
111         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112         label = self.display_label
113         # --- Sport profile + AI provider wiring (identical across segmenters) ---
114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
123         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
124         # needed and nothing is uploaded. Used to run the local detector standalone
125         # measure it against the Gemini path, or to avoid the cloud entirely). With
126         # FusionSegmenter the windows are still Gate-A/B-filtered via
127         # `_select_for_handoff`.
128         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
129         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
130         "gemini"))
131         if skip_ai:
132             model_name = "local-noai"
133             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
134             windows will "
135             "be emitted as rallies; no %s handoff, no API key needed.",
136             label, provider_name)
137         else:
138             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
139             "gemini-2.5-flash"))
140             api_key_env_var = f"{provider_name.upper()}_API_KEY"
141             api_key = os.environ.get(api_key_env_var)
142             if not api_key:
143                 logger.error(
144                     f"{api_key_env_var} environment variable is not set! "
145                     f"To run the {provider_name} handoff, set your API key. "
146                     f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
147                 )
148             return []
149         try:
150             provider = provider_registry.get(provider_name, api_key=api_key,
151             model_name=model_name)
152         except KeyError as e:
153             logger.error(str(e))
154             return []
155
156         video_duration = get_video_duration(video_path)
157         if video_duration <= 0:
158             logger.error("Invalid video duration.")
159             return []
160         fps, frame_width = self._probe_fps_width(video_path)
161
162         # --- Runner config + windowing thresholds ---
163         runner, runner_params = self._build_runner(idx_cfg)
164
165         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
166         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
167         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)

```

```

163         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
164         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
165         log_candidates = idx_cfg.get("log_yolo_candidates", True)
166         store_candidates = idx_cfg.get("store_yolo_candidates", True)
167
168         detection_params = {
169             "detector": self.name,
170             "velocity_thresh": velocity_thresh,
171             "merge_gap": merge_gap,
172             "min_action_window_duration": min_window_duration,
173             **runner_params,
174         }
175
176         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
177         ok, msg = runner.healthcheck()
178         if not ok:
179             logger.error("%s WSL environment not ready: %s", label, msg)
180             return []
181         logger.info("%s WSL env OK: %s", label, msg)
182
183         # --- Phase 2: trajectory -> candidate rally windows ---
184         # Trajectory detectors process the whole video in one pass (they carry
185         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
186         t_start = time.time()
187         out_dir = os.path.join("output", self.family)
188         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
189         if not csv_path:
190             logger.error("%s produced no trajectory; aborting.", label)
191             return []
192
193         points = runner.parse_trajectory_csv(csv_path)
194         logger.info("Parsed %d trajectory points (%d visible).",
195                     len(points), sum(1 for p in points if p.visible))
196
197         all_candidate_windows = runner.trajectory_to_action_windows(
198             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
199             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
200             min_window_duration=min_window_duration, chunk_index=0,
201         )
202         logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
203                     label, time.time() - t_start, len(all_candidate_windows))
204
205         if log_candidates:
206             for cw in all_candidate_windows:
207                 logger.info(f"[{label}] rally
208 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
209                             f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
210                             f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
211
212         # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
213         fusion
214         # segmenter adds net_crossings + person-cue features. Computed once, reused for
215         both
216         # persistence and the handoff gate below.
217         window_stats = [
218             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
219             idx_cfg)
220             for cw in all_candidate_windows
221         ]
222
223         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
224         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
225         if store_candidates:

```



```

222         for i, cw in enumerate(all_candidate_windows):
223             candidate_ids[i] = self.db.add_candidate_segment(
224                 video_id, detector=self.name,
225                 start_time=cw["start"], end_time=cw["end"],
226                 chunk_index=cw["chunk_index"], confidence=min(1.0,
227                     cw["peak_velocity"]),
228                 stats=window_stats[i], detection_params=detection_params,
229             )
230         if not all_candidate_windows:
231             return []
232
233         # --- Phase 3: AI provider handoff over padded candidate clips ---
234         # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
235         # candidates above stay the full superset ? only the served play_segments are
236         # gated.
237         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
238             idx_cfg)
239         ai_segments: List[dict] = []
240         if skip_ai:
241             # Fully-local: the selected candidate windows ARE the rallies ? emit them
242             # (no clip extraction, no upload). Boundaries are the local detector's; shot
243             # falls back to the duration-based estimate (no AI exchange count).
244             ai_segments = [
245                 {
246                     "start_time": all_candidate_windows[wi]["start"],
247                     "end_time": all_candidate_windows[wi]["end"],
248                     "shuttle_exchange_count": None,
249                     "shot_count_confidence": "LocalNoAI",
250                     "_candidate_id": candidate_ids[wi],
251                 }

```

21. user (2026-06-16T03:12:43.323Z)

(Bash completed with no output)

22. assistant (2026-06-16T03:12:46.064Z)

Let me check the config models and the DB schema:

23. user (2026-06-16T03:12:46.508Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):

```

```

19     court_green_lower: list[int] = Field(default=[35, 40, 40])
20     court_green_upper: list[int] = Field(default=[85, 255, 255])
21     court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73         keep_frames: bool = False
74         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75         min_free_gb: float = 0.0
76         # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
77         # on the fly and feed frames straight to the detector, skipping the slow per-frame
PNG

```

```

78         # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
79         # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is
lossless),
80         # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
it.
81         stream_video: bool = False
82
83
84     class FusionConfig(BaseModel):
85         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
86
87         Every default reproduces control behaviour: the fusion segmenter persists the
88         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
89         true ? the person-cue features, but it does NOT gate the served handoff unless a
90         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
91         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
92         supplied ? off-court persons (spectators / adjacent courts) are excluded.
93         """
94         # Which trajectory substrate seeds the windows (shuttle stays primary).
95         substrate: Literal["wasb", "tracknet"] = "wasb"
96         # Windowing velocity threshold for the fusion family (the engine reads
97         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
98         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
99         velocity_threshold: float = 0.02
100        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
101        # enabled ? so the default path never imports torch / touches the GPU.
102        cue_flags: dict[str, bool] = Field(default_factory=dict)
103        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
104        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
105        min_crossings: int = Field(default=0, ge=0)
106        # Served Gate B: when the person cue is on, drop windows with median in-court
players
107        # < this. 0 = OFF.
108        min_players: int = Field(default=0, ge=0)
109        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
110        court_polygon: Optional[list[list[float]]] = None
111        # Net line for the person two-sided-motion split (person features only ? the shuttle
112        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
113        net_axis: Literal["x", "y"] = "y"
114        net_position: float = Field(default=0.5, ge=0.0, le=1.0)
115
116
117     class IndexingConfig(BaseModel):
118         default_segmenter: str = "yolo_hybrid"
119         use_gpu: bool = True
120         motion_threshold: float = 0.08
121         min_segment_duration: float = 3.0
122         dead_time_merge_gap: float = 2.0
123         chunk_duration: float = 90.0
124         chunk_overlap: float = 10.0
125         detector_model: str = "fasterrcnn_resnet50_fpn"
126         detector_score_threshold: float = 0.5
127         yolo_velocity_threshold: float = 0.15
128         yolo_frame_skip: int = 3
129         yolo_tracker: str = "bytetrack.yaml"
130         yolo_prefetch_workers: int = 2
131         min_action_window_duration: float = 2.0
132         log_yolo_candidates: bool = True
133         store_yolo_candidates: bool = True

```

```

134     ai_handoff_padding: float = 2.0
135     ai_max_workers: int = 5
136     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
137     # chunks of a high-bitrate (4K) source blow past the provider upload cap
138     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
139     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
140     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
141     # Matches gemini_refine's --clip-scale-width default.
142     ai_chunk_scale_width: int = 1280
143     # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
144     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
145     # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
146     # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
147     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
148     skip_ai_handoff: bool = False
149     # Optional debug knob: trim every video to the first N seconds before processing.
150     debug_duration: Optional[float] = None
151
152     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
153     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
154     fusion: FusionConfig = Field(default_factory=FusionConfig)
155
156     @field_validator("default_segmenter")
157     @classmethod
158     def segmenter_must_be_registered(cls, v: str) -> str:
159         # Registry check happens at runtime in main/segmenter selection.
160         # Here we just allow any string for forward compatibility.
161         return v
162
163     @model_validator(mode="after")
164     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
165         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
166         # step makes `while t < duration: t += step` loop forever, growing memory before
any
167         # GPU work. Reject the bad config at load time instead.
168         if self.chunk_overlap >= self.chunk_duration:
169             raise ValueError(
170                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
171                 f"({self.chunk_duration}); otherwise chunking never advances."
172             )
173         return self
174
175
176     class OutputConfig(BaseModel):
177         default_highlights_name: str = "highlights.mp4"
178         db_name: str = "sports_indexer.db"
179         # Directory where the DB, highlight reels, and processed clips are written.
180         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
181         output_dir: str = "output"
182
183
184     class WatermarkConfig(BaseModel):
185         """Branding overlay burned into each highlight clip during the per-clip re-encode
186         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
187         (under `stitching.watermark` in config.json) to turn it off. The text is fully
188         configurable. The concat stage stays stream-copy (-c copy)."""
189         enabled: bool = True
190         text: str = "Generated by Khelsutra.guru"
191         logo_path: str = "" # optional transparent PNG overlay; "" = no logo

```

```

192     font_path: str = ""          # optional .ttf; "" = use the fontconfig `font` family
below
193     font: str = "Sans"          # fontconfig family used when font_path is empty
194     font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
195     opacity: float = Field(default=0.85, ge=0.0, le=1.0)
196     box: bool = True            # faint backing box behind the text for legibility
197     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
198     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
199
200
201     class StitchingConfig(BaseModel):
202         begin_padding: float = 0.5
203         end_padding: float = 0.5
204         use_cache: bool = False
205         min_shot_count: int = 1
206         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
207
208
209     class LoggingConfig(BaseModel):
210         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
211         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
212         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
213         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
214         # them to WARNING; set true to restore full chatter for request-flow debugging.
215         verbose_http: bool = False
216
217
218     class SecurityConfig(BaseModel):
219         # When set, /api/process_video_path must resolve under this directory (hosted/tenant
220         # mode). Default None preserves the single-user local 'any local file' workflow.
221         allowed_video_root: Optional[str] = None
222
223         # --- Chunked upload (B2, PR-2) ---
224         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
225         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
226         # contain upload_dir or /api/process will reject the uploaded paths.
227         upload_dir: str = "output/uploads"
228         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
229         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
230         max_upload_mb: int = 4096
231         max_part_mb: int = 95
232         allowed_upload_extensions: List[str] = ["mp4", "mov"]
233
234
235     class AppConfig(BaseModel):
236         """Root configuration object.
237
238         All runtime configuration should be accessed via an instance of this class
239         (or the loader) rather than raw dict lookups.
240         """
241
242         sport: Sport = "badminton"
243         ai_provider: str = "gemini"
244         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
245         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
246         ai_model: str = "gemini-flash-latest"
247
248         validation: ValidationConfig = Field(default_factory=ValidationConfig)
249         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
250         output: OutputConfig = Field(default_factory=OutputConfig)
251         stitching: StitchingConfig = Field(default_factory=StitchingConfig)

```

```

252     security: SecurityConfig = Field(default_factory=SecurityConfig)
253     logging: LoggingConfig = Field(default_factory=LoggingConfig)
254
255     # Allow extra keys during transition so existing config.json files do not break.
256     model_config = {
257         "extra": "allow",
258         "validate_assignment": True,
259     }
260
261     # Temporary migration shim for legacy dict-style access (config.get("section",
dict    {}).get(...)).
262     # Will be removed once all call sites (validators, segmenters, etc.) move to typed
263     # attribute access / the typed config. A nested *section* is returned as a plain
dict    dict
264     # via model_dump, so legacy chained access keeps working ? returning the Pydantic
265     # section model instead breaks it (a model has no .get), which bit every validator
on
266     # real runs. Scalar leaves are returned as-is.
267     def get(self, key: str, default: Any = None) -> Any:
268         _MISSING = object()
269         val: Any = _MISSING
270         if key in type(self).model_fields or hasattr(self, key):
271             val = getattr(self, key, _MISSING)
272         if val is _MISSING:
273             # Legacy convenience: find a leaf key inside any nested section. Sections
are
274             # discovered dynamically from the model fields, so a future top-level
section
275             # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
276             for fname in type(self).model_fields:
277                 section = getattr(self, fname, None)
278                 if isinstance(section, BaseModel) and hasattr(section, key):
279                     val = getattr(section, key)
280                     break
281             if val is _MISSING:
282                 return default
283             if isinstance(val, BaseModel):
284                 return val.model_dump(mode="json")
285             return val
286
287     def __getitem__(self, key: str) -> Any:
288         val = self.get(key)
289         if val is None and key not in type(self).model_fields:
290             # Re-raise like a real dict for unknown top-level
291             raise KeyError(key)
292         return val
293
294
295     # Convenience type alias for callers that want to annotate
296     Config = AppConfig
297

```

24. user (2026-06-16T03:12:47.057Z)

(Bash completed with no output)

25. assistant (2026-06-16T03:12:48.714Z)

Now let me find the DB schema and candidate_segments table:

26. user (2026-06-16T03:12:52.001Z)

./../.claude/worktrees/agent-a771ff1d85e18f2ed/backend/infrastructure/database.py

```
./claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_database.py
./claude/worktrees/bhi-execpolicy/backend/infrastructure/database.py
./claude/worktrees/bhi-execpolicy/tests/test_database.py
./claude/worktrees/local-noai/backend/infrastructure/database.py
```

27. user (2026-06-16T03:12:52.309Z)

(Bash completed with no output)

28. user (2026-06-16T03:12:54.210Z)

```
./claude/worktrees/agent-a771ff1d85e18f2ed/backend/infrastructure/database.py
./claude/worktrees/agent-a771ff1d85e18f2ed/backend/infrastructure/execution_policy.py
./claude/worktrees/bhi-execpolicy/backend/infrastructure/database.py
./claude/worktrees/bhi-execpolicy/backend/infrastructure/execution_policy.py
./claude/worktrees/local-noai/backend/infrastructure/database.py
./claude/worktrees/local-noai/backend/infrastructure/execution_policy.py
./backend/infrastructure/database.py
./backend/infrastructure/execution_policy.py
```

29. user (2026-06-16T03:12:55.867Z)

```
1     import sqlite3
2     import json
3     import logging
4     from contextlib import contextmanager
5     from typing import List, Dict, Any, Optional, Tuple
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
16             """Transaction scope that also CLOSES the connection.
17
18             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19             never closes, so every call leaked its connection to GC (lingering file
20             handles on the WAL sidecars under Windows). Internal methods use this.
21             """
22             conn = self._get_connection()
23             try:
24                 with conn:
25                     yield conn
26             finally:
27                 conn.close()
28
29         def _get_connection(self):
30             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31             # adds concurrent writers (a swallowed 'database is locked' silently drops
32             segments).
33             conn = sqlite3.connect(self.db_path, timeout=30.0)
34             conn.row_factory = sqlite3.Row
35             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
36             conn.execute("PRAGMA foreign_keys = ON")
37             # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
38             conn.execute("PRAGMA busy_timeout = 30000")
39             try:
40                 conn.execute("PRAGMA journal_mode = WAL")
41             except sqlite3.OperationalError:
42                 pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
```

```

42         return conn
43
44     def _init_db(self):
45         """Initializes tables for videos, play_segments, and events."""
46         with self._connect() as conn:
47             cursor = conn.cursor()
48
49             # Videos Table
50             cursor.execute("""
51                 CREATE TABLE IF NOT EXISTS videos (
52                     id TEXT PRIMARY KEY,
53                     filepath TEXT NOT NULL,
54                     processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
55                     status TEXT NOT NULL,
56                     validation_results TEXT
57                 )
58             """)
59
60             # Play Segments Table (Extensible metadata JSON)
61             cursor.execute("""
62                 CREATE TABLE IF NOT EXISTS play_segments (
63                     id INTEGER PRIMARY KEY AUTOINCREMENT,
64                     video_id TEXT NOT NULL,
65                     start_time REAL NOT NULL,
66                     end_time REAL NOT NULL,
67                     duration REAL NOT NULL,
68                     server TEXT,
69                     receiver TEXT,
70                     metadata TEXT,
71                     FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
72                 )
73             """)
74
75             # Events Table
76             cursor.execute("""
77                 CREATE TABLE IF NOT EXISTS events (
78                     id INTEGER PRIMARY KEY AUTOINCREMENT,
79                     segment_id INTEGER NOT NULL,
80                     timestamp REAL NOT NULL,
81                     type TEXT NOT NULL,
82                     player TEXT,
83                     details TEXT,
84                     FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
85             CASCADE
86                 )
87             """)
88
89             # Versioned migrations live here. PRAGMA user_version is the schema
90             # contract ? bump it for every change and add an ordered `if version < N`
91             # block below. Tests assert the expected version, so accidental drift fails
92             CI.
93             version = cursor.execute("PRAGMA user_version").fetchone()[0]
94
95             if version < 1:
96                 # v1: raw candidate detections (pre-confirmation), detector-agnostic.
97                 # Common fields are columns; detector-specific metrics go in `stats`
98                 JSON,
99                 # so a new segmenter reuses this table by setting its own
100                 `detector`/`stats`.
101                 cursor.execute("""
102                     CREATE TABLE IF NOT EXISTS candidate_segments (
103                         id INTEGER PRIMARY KEY AUTOINCREMENT,
104                         video_id TEXT NOT NULL,
105                         detector TEXT NOT NULL,
106                         chunk_index INTEGER,

```



```

103         start_time REAL NOT NULL,
104         end_time REAL NOT NULL,
105         duration REAL NOT NULL,
106         confidence REAL,
107         stats TEXT,
108         detection_params TEXT,
109         confirmed_segment_id INTEGER,
110         human_label TEXT,
111         notes TEXT,
112         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
113         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
114         FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
115     )
116     """)
117     cursor.execute("""
118         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
119         ON candidate_segments(video_id, detector)
120     """)
121     cursor.execute("PRAGMA user_version = 1")
122
123     if version < 2:
124         # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
125         # /api/process request. `status` is the EXECUTION state of the job
126         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
127         # that failed validation) lives inside `result` JSON as
128         # `result` is the full sync-era response dict (incl. report paths), so
the
129         # observability report is the job's artifact, per the plan.
130         cursor.execute("""
131             CREATE TABLE IF NOT EXISTS jobs (
132                 id TEXT PRIMARY KEY,
133                 video_path TEXT NOT NULL,
134                 video_id TEXT,
135                 segmenter TEXT,
136                 player_pool TEXT,
137                 max_frames INTEGER,
138                 status TEXT NOT NULL DEFAULT 'queued',
139                 progress TEXT,
140                 error TEXT,
141                 result TEXT,
142                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
143                 started_at TIMESTAMP,
144                 finished_at TIMESTAMP
145             )
146             """)
147         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
148         cursor.execute("PRAGMA user_version = 2")
149
150     if version < 3:
151         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
152         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
153         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
154         # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
155         # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
156         cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
157         cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
158         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,

```

```

next_poll_at)")
159         cursor.execute("PRAGMA user_version = 3")
160
161     if version < 4:
162         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
163         # `user_id` to videos + candidate_segments so a future per-user / multi-
164         # path can scope rows to an owner. **NULL = local install = byte-
165         # identical to
166         # today** ? every existing row stays NULL and every existing query
167         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
168         # survive.
169         # Additive ONLY: no served-behavior change rides on this slice.
170         cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
171         cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
172         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
173         # of index churn while making the eventual per-user lookups cheap.
174         cursor.execute(
175             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
176             "WHERE user_id IS NOT NULL")
177         cursor.execute(
178             "CREATE INDEX IF NOT EXISTS idx_candidates_user "
179             "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
180         cursor.execute("PRAGMA user_version = 4")
181
182     if version < 5:
183         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
184         # (user_id,
185         # version): the resolved per-user `fusion_config` overlay + an INLINE
186         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
187         # evidence
188         # and provenance that earned it. `is_active` pins the one row the
189         # serving bridge
190         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
191         # This slice
192         # only persists/loads it ? NOTHING reads it on the served path yet.
193         #
194         # user_id          ? owner of the model (NULL allowed = the local
195         # install)
196         # version          ? monotonic per-user model version
197         # baseline_version  ? the shared baseline this was fit against
198         # (upgrade switch)
199         # feature_schema_hash ? fingerprint of the feature set; MINOR-vs-
200         # MAJOR re-fit gate
201         # fusion_config     ? JSON per-user FusionConfig overlay (cue_flags
202         # / thresholds)
203         # classifier_json   ? JSON LogisticRegression.to_dict() (None =
204         # config-only model)
205         # promotion_evidence ? JSON per_user_gate PromotionDecision (why it
206         # was promoted)
207         # n_videos_at_fit   ? held-out-user corpus size when fit (min_n
208         # trust floor input)
209         # is_active         ? the one resolved row for this user (partial-
210         # unique below)
211         cursor.execute("""
212             CREATE TABLE IF NOT EXISTS user_models (
213                 id INTEGER PRIMARY KEY AUTOINCREMENT,
214                 user_id TEXT,
215                 version INTEGER NOT NULL DEFAULT 1,
216                 baseline_version TEXT,
217                 feature_schema_hash TEXT,
218                 fusion_config TEXT,

```

```

205             classifier_json TEXT,
206             promotion_evidence TEXT,
207             n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
208             is_active INTEGER NOT NULL DEFAULT 0,
209             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
210         )
211     """
212     # One model version per user (NULLs compare distinct in SQLite, so the
local
213     # install can still carry multiple versioned rows; the active-row guard
below
214     # is what enforces a single served model).
215     cursor.execute("""
216         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
217         ON user_models(user_id, version)
218     """)
219     # At most ONE active model per user: a partial-unique index over the
active rows.
220     # (SQLite treats NULL user_id rows as distinct, so the local install's
single
221     # active row is still guarded ? there is one local user.)
222     cursor.execute("""
223         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
224         ON user_models(user_id) WHERE is_active = 1
225     """)
226     cursor.execute("PRAGMA user_version = 5")
227     # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6
228
229     conn.commit()
230
231     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
232         """Register or update a video row WITHOUT touching its child segments.
233
234         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
235         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
236         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
237         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
238         row in place; clearing segments is now an explicit, deliberate operation
239         (clear_video_data / prune_segments).
240         """
241         try:
242             with self._connect() as conn:
243                 conn.cursor().execute(
244                     "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
245                     "ON CONFLICT(id) DO UPDATE SET "
246                     "filepath=excluded.filepath, status=excluded.status, "
247                     "validation_results=excluded.validation_results",
248                     (video_id, filepath, status, json.dumps(validation_results))
249                 )
250                 conn.commit()
251             return True
252         except Exception as e:
253             logger.error(f"Failed to add video: {e}")
254             return False
255
256     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
257         """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
258

```

```

259         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
260         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
261         """
262         with self._connect() as conn:
263             cur = conn.cursor()
264             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
265                             (video_id,)).fetchone()[0]
266             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
267                             (video_id,)).fetchone()[0]
268             return int(p), int(c)
269
270     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
271                       play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
272                       cand_after: Optional[int] = None) -> None:
273         """Delete play/candidate segments by id range (events cascade from
play_segments).
274
275         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
276         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
277         and keep the OLD ones when the run failed/produced nothing (`*_after`).
278         """
279         with self._connect() as conn:
280             cur = conn.cursor()
281             if play_upto is not None:
282                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
283             if play_after is not None:
284                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
285             if cand_upto is not None:
286                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
287             if cand_after is not None:
288                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
289             conn.commit()
290
291     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
292         try:
293             with self._connect() as conn:
294                 cursor = conn.cursor()
295                 cursor.execute(
296                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
297                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
298                 )
299                 conn.commit()
300                 return cursor.lastrowid

```

30. assistant (2026-06-16T03:12:58.142Z)

Let me continue reading the DB schema to see the add_candidate_segment method:

31. user (2026-06-16T03:12:58.791Z)

```

300         return cursor.lastrowid
301     except Exception as e:
302         logger.error(f"Failed to add play segment: {e}")

```

```

303         return None
304
305     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
306         try:
307             with self._connect() as conn:
308                 conn.cursor().execute(
309                     "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
310                     (segment_id, timestamp, event_type, player, json.dumps(details or
{})))
311                 )
312                 conn.commit()
313                 return True
314         except Exception as e:
315             logger.error(f"Failed to add event: {e}")
316             return False
317
318     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
319                               chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
320                               stats: Optional[Dict[str, Any]] = None,
321                               detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
322         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
323         detector-specific dicts stored as JSON. Returns the new candidate id, or
324         None."""
325         try:
326             with self._connect() as conn:
327                 cursor = conn.cursor()
328                 cursor.execute(
329                     """INSERT INTO candidate_segments
330                     (video_id, detector, chunk_index, start_time, end_time, duration,
331                     confidence, stats, detection_params)
332                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
333                     (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
334                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
335                 )
336                 conn.commit()
337                 return cursor.lastrowid
338         except Exception as e:
339             logger.error(f"Failed to add candidate segment: {e}")
340             return None
341
342     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
343         """Record that a candidate detection was confirmed as a given play_segment."""
344         try:
345             with self._connect() as conn:
346                 conn.cursor().execute(
347                     "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
348                     (segment_id, candidate_id)
349                 )
350                 conn.commit()
351                 return True
352         except Exception as e:
353             logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
354             return False
355
356     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
only_unlabeled: bool = False) -> List[Dict[str, Any]]:

```

```

357         """Fetch candidate detections for the annotation / fine-tuning workflow.
358         `stats` and `detection_params` are deserialized from JSON."""
359         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
360         params: List[Any] = [video_id]
361
362         if detector:
363             query += " AND detector = ?"
364             params.append(detector)
365         if only_unlabeled:
366             query += " AND human_label IS NULL"
367
368         query += " ORDER BY start_time"
369
370         candidates = []
371         with self._connect() as conn:
372             rows = conn.cursor().execute(query, params).fetchall()
373             for row in rows:
374                 d = dict(row)
375                 d["stats"] = json.loads(d["stats"] or "{}")
376                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
377                 candidates.append(d)
378         return candidates
379
380     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
381         with self._connect() as conn:
382             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
383 (video_id,)).fetchone()
384             if row:
385                 d = dict(row)
386                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
387                 return d
388             return None
389
390     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
391         """
392         Dynamically queries play segments based on filters:
393         - duration_min: float
394         - server: string
395         - receiver: string
396         - event_type: string (e.g. smash, foul)
397         """
398         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
399         params = [video_id]
400
401         if filters.get("duration_min"):
402             query += " AND r.duration >= ?"
403             params.append(filters["duration_min"])
404
405         if filters.get("server"):
406             query += " AND r.server = ?"
407             params.append(filters["server"])
408
409         if filters.get("receiver"):
410             query += " AND r.receiver = ?"
411             params.append(filters["receiver"])
412
413         if filters.get("event_type"):
414             # Check if there is an event of a specific type inside this segment
415             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
416             params.append(filters["event_type"])
417
418         segments_list = []
419         with self._connect() as conn:

```

```

419         cursor = conn.cursor()
420         rows = cursor.execute(query, params).fetchall()
421
422         for row in rows:
423             r_dict = dict(row)
424             r_id = r_dict["id"]
425             r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
426
427             # Fetch associated events
428             event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
429             r_dict["events"] = []
430             for er in event_rows:
431                 e_dict = dict(er)
432                 e_dict["details"] = json.loads(e_dict["details"] or "{}")
433                 r_dict["events"].append(e_dict)
434
435             segments_list.append(r_dict)
436
437         return segments_list
438
439     def clear_video_data(self, video_id: str):
440         with self._connect() as conn:
441             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
442             conn.commit()
443
444     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
445     # Writers follow the repo convention (swallow + logger.error, return bool); the
446     # worker logs loudly on a failed transition so a job can't silently wedge.
447
448     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
449                   player_pool: Optional[List[str]] = None,

```

32. user (2026-06-16T03:12:59.244Z)

(Bash completed with no output)

33. assistant (2026-06-16T03:13:00.950Z)

Now let me check person_detector to understand its windowing and feature computation:

34. user (2026-06-16T03:13:01.594Z)

```

100     def detect_and_track(self, frame: Any, tracker: Any,
101                          score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
102         """Detect persons in a frame and assign persistent track IDs.
103
104         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
105         This replaces the old ultralytics ``model.track(...)`` call, which bundled
106         detection AND association: torchvision does the detection, supervision's
107         ByteTrack does the association.
108         """
109         import supervision as sv
110
111         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
112         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
113         tensor = torch.from_numpy(rgb).permute(2, 0,
1).float()).div(255.0).to(self.device)
114         with torch.no_grad():
115             outputs = self.model([tensor])
116
117         out = outputs[0]
118         boxes = out["boxes"].detach().cpu().numpy()

```

```

119         labels = out["labels"].detach().cpu().numpy()
120         scores = out["scores"].detach().cpu().numpy()
121
122         # Keep only confident person detections.
123         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
124         if not mask.any():
125             return []
126
127         # ByteTrack requires confidence to be populated (it filters on it internally).
128         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
129         tracked = tracker.update_with_detections(dets)
130
131         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
132         for i in range(len(tracked)):
133             tid = tracked.tracker_id[i]
134             if tid is None:
135                 continue
136             x1, y1, x2, y2 = tracked.xyxy[i]
137             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
138         return result
139
140     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
141                                         velocity_thresh: float, merge_gap: float,
142                                         frame_skip: int = 3,
143                                         chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
144         """
145         Runs person detection + tracking on a video chunk and extracts high-motion
146         action windows. Returns a list of dicts in the full video's timeframe, each
with:
147             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
148             track_id_count, chunk_index.
149
150         Args:
151             frame_skip: Process every Nth frame (1 = every frame). Higher values
152                         dramatically reduce inference cost with minimal quality loss.
153             chunk_index: Index of the source chunk (recorded on each window for
traceability).
154         """
155         cap = cv2.VideoCapture(chunk_path)
156         if not cap.isOpened():
157             logger.error(f"Could not open {chunk_path}")
158             return []
159
160         fps = cap.get(cv2.CAP_PROP_FPS)
161         if fps <= 0:
162             fps = 30.0
163
164         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
165         if frame_width <= 0:
166             frame_width = 1280.0 # Sensible fallback
167
168         # Effective sampling rate after frame-skipping
169         effective_fps = fps / max(frame_skip, 1)
170
171         # A fresh tracker per chunk ? track IDs only need to be consistent within a
172         # chunk (windows are computed per chunk, then merged across chunks by time).
173         tracker = self.make_tracker(effective_fps)
174         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
175
176         # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
177         # so windows can be enriched with motion stats for logging + fine-tuning.
178         active_frames = []

```



```

179     frame_idx = 0
180     track_history: Dict[int, Tuple[float, float, float, float]] = {}
181
182     while True:
183         ret, frame = cap.read()
184         if not ret:
185             break
186
187         # Skip frames for performance ? at 30fps, every 3rd frame still
188         # gives ~10 samples/second, plenty for human-scale velocity.
189         if frame_idx % max(frame_skip, 1) != 0:
190             frame_idx += 1
191             continue
192
193         # Detect persons + assign persistent track IDs.
194         detections = self.detect_and_track(frame, tracker, score_thresh)
195
196         frame_peak_velocity = 0.0
197         frame_active_ids = set()
198         current_ids = set()
199
200         for track_id, bbox in detections:
201             current_ids.add(track_id)
202
203             if track_id in track_history:
204                 prev_bbox = track_history[track_id]
205                 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
206
207                 if velocity > velocity_thresh:
208                     frame_peak_velocity = max(frame_peak_velocity, velocity)
209                     frame_active_ids.add(track_id)
210
211                 track_history[track_id] = bbox
212
213             # Prune stale IDs that are no longer visible ? prevents false
214             # velocity spikes when the tracker reassigns an old ID.
215             track_history = {k: v for k, v in track_history.items() if k in current_ids}
216
217             if frame_active_ids:
218                 active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
219
220             frame_idx += 1
221
222         cap.release()
223
224         # Group active frames into continuous windows
225         if not active_frames:
226             return []
227
228         max_gap_frames = int(merge_gap * fps)
229
230         def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
231             velocities = [v for _, v, _ in group]
232             track_ids = set().union(*[ids for _, _, ids in group])
233             return {
234                 "start": group[0][0] / fps + chunk_start,
235                 "end": group[-1][0] / fps + chunk_start,
236                 "active_frame_count": len(group),
237                 "peak_velocity": max(velocities),
238                 "mean_velocity": sum(velocities) / len(velocities),
239                 "track_id_count": len(track_ids),
240                 "chunk_index": chunk_index,
241             }
242
243         windows = []

```

```

243         group = [active_frames[0]]
244         for af in active_frames[1:]:
245             if af[0] - group[-1][0] <= max_gap_frames:
246                 group.append(af)
247             else:
248                 windows.append(_finalize(group))
249                 group = [af]
250         windows.append(_finalize(group))
251
252         # Filter out extremely short action windows
253         min_window_duration = self.config.get("indexing",
254         {}).get("min_action_window_duration", 2.0)
255         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
256
257         return windows
258
259     def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
260                             frame_skip: int = 3) -> Dict[str, Any]:
261         """Per-frame person detections over `[start_frame, end_frame]` of the ORIGINAL
262         video.
263
264         Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
265         ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
266         discards the
267         per-frame boxes), this returns the raw per-frame detections so the fusion
268         features can
269         align them with the shuttle trajectory. It reads the source by **absolute frame
270         index**
271         (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
272         match
273         the WASB trajectory's ``frame`` exactly ? that alignment is what
274         ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
275
276         Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
277         "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
278         seek
279         accuracy varies by codec (it lands near the requested keyframe then decodes
280         forward);
281         treat the indices as ±a few frames on real footage ? fine for window-level
282         features.
283
284         """
285         self.lazy_load_model()
286         cap = cv2.VideoCapture(video_path)
287         if not cap.isOpened():
288             logger.error(f"Could not open {video_path}")
289             return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
290
291         fps = cap.get(cv2.CAP_PROP_FPS)
292         if fps <= 0:
293             fps = 30.0
294         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
295         frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
296         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
297         0.5)
298
299         step = max(frame_skip, 1)
300         tracker = self.make_tracker(fps / step)
301
302         start_frame = max(0, int(start_frame))
303         frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
304         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
305         frame_idx = start_frame
306         while frame_idx <= end_frame:
307             ret, frame = cap.read()
308             if not ret:
309                 break

```

```
298         if (frame_idx - start_frame) % step == 0:
299             frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
```

35. assistant (2026-06-16T03:13:03.437Z)

Now let me check the wasb_runner to see the full implementation there:

36. user (2026-06-16T03:13:03.949Z)

```
1      """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4          1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5          2. copies wasb_infer.py into the WASB repo's src/ dir,
6          3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7          4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9      So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10     with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11     """
12     import os
13     import shlex
14     import logging
15     import subprocess
16     from dataclasses import dataclass
17     from typing import Dict, Any, Optional, Tuple
18
19     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
21     TrackNetRunner.
22     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
23     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
24     wsl_tilde_quote
25
26     logger = logging.getLogger(__name__)
27
28     # Windows path to the inference wrapper we stage into WSL.
29     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
30
31     @dataclass
32     class WasbConfig:
33         """Resolved settings for a WASB WSL run (built from config.json ->
34         indexing.wasb)."""
35         distro: str = "Ubuntu"
36         repo_dir: str = "~/models/WASB-SBDT"
37         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
38         conda_env: str = "wasb"
39         weights_path: str = "~/models/WASB-
40         SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
41         sport: str = "badminton"
42         wsl_stage_dir: str = "~/clips"
43         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
44         keep_frames: bool = False # retain staged video + frame cache after success (debug
45         only)
46         min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
47         off)
48         # FAST PATH (default OFF until verified): decode the staged video on the fly inside
49         # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
50         # extraction that dominates a long clip. Output is bit-identical to the PNG path
51         # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
52         it.
53         stream_video: bool = False
54
55     @classmethod
```

```

50     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51         w = (idx_cfg or {}).get("wasb", {}) or {}
52         return cls(
53             distro=w.get("wsl_distro", "Ubuntu"),
54             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56             conda_env=w.get("conda_env", "wasb"),
57             weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),
64             stream_video=bool(w.get("stream_video", False)),
65         )
66
67
68     class WasbRunner(WslCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):
70             self.cfg = cfg
71
72         def healthcheck(self) -> Tuple[bool, str]:
73             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74             cmd = (f"conda activate {self.cfg.conda_env} && "
75                  f"python -c \"import torch; print('torch', torch.__version__, "
76                  f"'cuda', torch.cuda.is_available())\"")
77             try:
78                 res = self._wsl_bash(cmd)
79             except FileNotFoundError:
80                 return False, "`wsl` not found ? Windows + WSL2 required."
81             except subprocess.TimeoutExpired:
82                 return False, "WSL healthcheck timed out."
83             out = (res.stdout or "").strip()
84             if res.returncode != 0:
85                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86             return True, out
87
88         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89             src_mnt = to_wsl_mnt_path(video_win_path)
90             reason = self.wsl_source_unreadable_reason(src_mnt)
91             if reason:
92                 logger.error("Cannot stage video into WSL: %s", reason)
93                 return None
94             dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
96             try:
97                 res = self._wsl_bash(cmd)
98             except subprocess.TimeoutExpired:
99                 logger.error("Staging video into WSL timed out.")
100                 return None
101             if res.returncode != 0 or "OK" not in (res.stdout or ""):
102                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
103                 return None
104             return dest
105
106         def _stage_infer_script(self) -> bool:
107             """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
108             src_mnt = to_wsl_mnt_path(_INFER_WIN)
109             cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
110             res = self._wsl_bash(cmd)

```

```

111         return res.returncode == 0 and "OK" in (res.stdout or "")
112
113     def run_predict(self, video_win_path: str, output_win_dir: str,
114                    video_id: Optional[str] = None) -> Optional[str]:
115         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
116
117         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
118         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
119         can never reuse each other's staged frames, resumable cache, or CSV.
120         """
121         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
122         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
123         # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
124         # the CSV landed inside WSL while Windows polled the repo-relative path.
125         output_win_dir = os.path.abspath(output_win_dir)
126         os.makedirs(output_win_dir, exist_ok=True)
127         # Normalize for cross-platform basename (tests use Windows paths on Linux).
128         video_win_path = str(video_win_path).replace("\\", "/")
129         base = os.path.basename(video_win_path)
130         stem, ext = os.path.splitext(base)
131         # Unique, content-derived key: same filename + different bytes -> different key.
132         key = f"{stem}__{video_id[:12]}" if video_id else stem
133         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
134         expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
135
136         if not self._stage_infer_script():
137             logger.error("Could not stage wasb_infer.py into WSL.")
138             return None
139         staged_video = self._stage_video(video_win_path, f"{key}{ext}")
140         if staged_video is None:
141             return None
142         frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144         # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145         # WSL stage filesystem is already low so the user can clean up before it fills.
146         # (Streaming never materializes the PNGs, so the guard only matters off the fast
147         path.)
148         if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
149             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
150             if free is not None and free < self.cfg.min_free_gb:
151                 logger.warning(
152                     "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
153                     extraction "
154                     "may fill the disk ? consider running tools/cleanup_caches.py.",
155                     free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
156
157         # Fast path: stream-decode the video in-process (no PNG extraction). Output is
158         # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
159         clip.
160         if self.cfg.stream_video:
161             frame_args = "--stream-video "
162         else:
163             frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
164         cmd = (
165             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
166             f"python wasb_infer.py "
167             f"--video {wsl_tilde_quote(staged_video)} "
168             f"{frame_args}"
169             f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
170             f"--sport {shlex.quote(self.cfg.sport)} "
171             f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}")
172         )
173         logger.info("Running WASB inference on %s ...", base)
174         try:
175             res = self._wsl_bash(cmd)

```

```

173         except subprocess.TimeoutExpired:
174             logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175             return None
176         if res.returncode != 0:
177             logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178             return None
179         if not os.path.exists(expected_csv_win):
180             logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181             return None
182         logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184         # Success ? the trajectory CSV is the durable output; the staged video copy and
185         # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186         # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187         # so the frame cache + detector cache remain for a resumable re-run.
188         # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189         if not self.cfg.keep_frames:
190             stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191             logger.info("Cache hygiene: removing staged video%s for %s "
192                         "(set indexing.wasb.keep_frames=true to retain).",
193                         "" if self.cfg.stream_video else " + frame cache", key)
194             self._wsl_rm_rf(stale)
195             return expected_csv_win
196
197         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

37. assistant (2026-06-16T03:13:06.292Z)

Perfect! Now let me look for any existing analysis of fps sensitivity and search for any test/eval files that compare 30 vs 60 fps:

38. user (2026-06-16T03:13:07.388Z)

```

.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/audio/e1_probe.py:    ap.add_argument("--
fps", type=float, default=59.94)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/calibrate_wasb.py:        --labels
"golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/calibrate_wasb.py:    ap.add_argument("--
fps", type=float, default=59.94)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/eval_partial_labels.py:        --fps
59.94 --frame-width 1920
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/eval_partial_labels.py:
ap.add_argument("--fps", type=float, default=59.94)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/export_wasb_segments.py:        --labels
"Annotation Setup/golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/export_wasb_segments.py:
ap.add_argument("--fps", type=float, default=59.94)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/gemini_refine.py:        --fps 59.94
--frame-width 1920 --out output/adarsh_gemini.csv [--limit 4]
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/gemini_refine.py:    ap.add_argument("--
fps", type=float, default=59.94)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/features/rally_seq.py:59.94fps than 30fps) is
exactly the train/serve-skew class this placement prevents.
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/features/rally_seq.py:    speed read ~2x
smaller at 59.94 fps than at 30 fps, confounding cross-footage transfer
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/person_detector.py:

```

```
# Skip frames for performance ? at 30fps, every 3rd frame still
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/validators/resolution_fps.py:
return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate (e.g. 30fps)
requirements."
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/reporting/harvest.py:      # used its internal
fallback (30fps / 1280px) ? surface that as the footgun it is,
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/reporting/harvest.py:      vm["fps"] =
30.0 # matches the runners' fallback (_probe_fps_width -> 30.0 / 1280)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/tools/export.py:      # EDL is HH:MM:SS:FF
(assume 30fps for the timecode display; real edit rate
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/tools/export.py:      f = int((t -
int(t)) * 30) # 30 fps display
.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_calibrate_wasb.py:      for i in range(0,
30):      # 0.0-2.9s @10fps: shuttle moving fast
.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_person_detector.py:      """A
cv2.VideoCapture mock yielding `num_frames` readable frames at 30fps."""
.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_reporting_harvest.py:      assert
env.video_meta.fps == 59.94
```

39. assistant (2026-06-16T03:13:09.210Z)

Excellent! Let me read that rally_seq.py file that has the fps sensitivity note:

40. user (2026-06-16T03:13:09.816Z)

```
"""Per-frame rally features over a shuttle/ball trajectory ? the rally_seq feature schema.
```

THE single source of truth for the Gen-0 TAL feature extraction (ADR-008): the training harness (training/) and the serving path import THIS module, so train == serve by construction. The fps-normalization bug fixed on 2026-06-10 (speed read ~2x smaller at 59.94fps than 30fps) is exactly the train/serve-skew class this placement prevents.

```
Pure numpy/scipy ? importable without torch/cv2. Any change to the feature semantics
MUST bump FEATURE_SCHEMA_VERSION; artifact manifests record the version they were
trained with and the serving loader refuses a mismatch.
"""
```

41. assistant (2026-06-16T03:13:12.023Z)

Great! Let me look at that rally_seq.py file more completely:

42. user (2026-06-16T03:13:12.354Z)

```
1      """Per-frame rally features over a shuttle/ball trajectory ? the rally_seq feature
schema.
2
3      THE single source of truth for the Gen-0 TAL feature extraction (ADR-008): the training
4      harness (training/) and the serving path import THIS module, so train == serve by
5      construction. The fps-normalization bug fixed on 2026-06-10 (speed read ~2x smaller at
6      59.94fps than 30fps) is exactly the train/serve-skew class this placement prevents.
7
8      Pure numpy/scipy ? importable without torch/cv2. Any change to the feature semantics
9      MUST bump FEATURE_SCHEMA_VERSION; artifact manifests record the version they were
10     trained with and the serving loader refuses a mismatch.
11     """
12     from __future__ import annotations
13
14     from typing import Dict
15
16     import numpy as np
17     from scipy.ndimage import uniform_filter1d, maximum_filter1d
18
19     # Bump on ANY semantic change to build_frame_arrays/features (names, normalization,
20     # windowing, occlusion rule). Artifacts pin this; the loader hard-fails on mismatch.
```

```

21     FEATURE_SCHEMA_VERSION = 1
22
23     FEAT_NAMES = ["vis_rate", "spd_mean", "spd_max", "cross_rate", "x_std", "y_std"]
24
25     # Default windowing ? part of the schema (manifests record the values actually used).
26     DEFAULT_WIN_SEC = 0.75
27     DEFAULT_STRIDE_SEC = 0.1
28
29     FEATURE_SCHEMA = {
30         "id": "rally_seq",
31         "version": FEATURE_SCHEMA_VERSION,
32         "feat_names": FEAT_NAMES,
33         "win_sec": DEFAULT_WIN_SEC,
34         "stride_sec": DEFAULT_STRIDE_SEC,
35         # Explicit because its absence was a real bug: speed is fraction-of-frame-width
36         # PER SECOND (* fps), matching the heuristic windowing brain.
37         "fps_semantics": "per-second",
38         "occlusion_rule": "speed/crossings not computed across gaps > max(2,
round(0.25*fps)) frames",
39         "input": "trajectory points (frame, visible, x, y) + REQUIRED runtime metadata {fps,
frame_width}",
40     }
41
42
43     def _spread(a: np.ndarray) -> float:
44         return float(np.percentile(a, 95) - np.percentile(a, 5)) if a.size > 1 else 0.0
45
46
47     def build_frame_arrays(points, frame_width: float, fps: float = 30.0) -> Dict:
48         """Per-frame trajectory arrays + per-visible-frame speed and net-crossing events.
49
50         Speed is normalized to fraction-of-frame-width PER SECOND (`* fps`), matching the
51         heuristic windowing brain ? previously it was per-frame, so the same physical
shuttle
52         speed read ~2x smaller at 59.94 fps than at 30 fps, confounding cross-footage
transfer
53         on a corpus that mixes frame rates. Speed/crossings are also not computed across a
long
54         occlusion gap (mirrors the heuristic resetting on invisibility)."""
55         pts = sorted(points, key=lambda p: p.frame)
56         N = (pts[-1].frame + 1) if pts else 1
57         vis = np.zeros(N)
58         x = np.zeros(N)
59         y = np.zeros(N)
60         for p in pts:
61             if 0 <= p.frame < N and p.visible and p.x >= 0:
62                 vis[p.frame] = 1.0
63                 x[p.frame] = float(p.x)
64                 y[p.frame] = float(p.y)
65
66         mask = vis > 0
67         play_is_x = _spread(x[mask]) >= _spread(y[mask])
68         coord = x if play_is_x else y
69         net = float(np.median(coord[mask])) if mask.any() else 0.0
70         deadband = 0.06 * (_spread(coord[mask]) if mask.sum() > 1 else 1.0)
71
72         speed = np.zeros(N)
73         spd_mask = np.zeros(N)
74         cross = np.zeros(N)
75         max_gap = max(2, round(0.25 * fps)) # frames; a longer gap = occlusion break, not
motion
76         prev = None
77         for p in pts:
78             f = p.frame
79             if not (0 <= f < N and vis[f]):

```



```

80         continue
81     if prev is not None:
82         df = f - prev
83         if 0 < df <= max_gap:
84             speed[f] = float(np.hypot(x[f] - x[prev], y[f] - y[prev]) / frame_width
/ df * fps)
85             spd_mask[f] = 1.0
86             c, pc = coord[f] - net, coord[prev] - net
87             if abs(c) > deadband and abs(pc) > deadband and (c > 0) != (pc > 0):
88                 cross[f] = 1.0
89         prev = f
90     return dict(N=N, vis=vis, x=x, y=y, speed=speed, spd_mask=spd_mask, cross=cross,
fw=frame_width)
91
92
93     def features(arr: Dict, fps: float, win_sec: float = DEFAULT_WIN_SEC,
94                 stride_sec: float = DEFAULT_STRIDE_SEC):
95         """Windowed per-frame features sampled at a stride -> (X[n_samples,6],
times[n_samples])."""
96         N = arr["N"]
97         fw = arr["fw"]
98         size = max(3, int(win_sec * fps) * 2 + 1)
99         vis, speed, spd_mask, cross, x, y = (arr[k] for k in ("vis", "speed", "spd_mask",
"cross", "x", "y"))
100
101         n = uniform_filter1d(vis, size, mode="constant") # mean
visibility (== vis_rate)
102         sm = uniform_filter1d(spd_mask, size, mode="constant")
103         spd_mean = uniform_filter1d(speed, size, mode="constant") / np.maximum(sm, 1e-9)
104         spd_max = maximum_filter1d(speed, size, mode="constant")
105         cross_rate = uniform_filter1d(cross, size, mode="constant") * fps # crossings
/ second
106         xm = uniform_filter1d(x * vis, size, mode="constant") / np.maximum(n, 1e-9)
107         x2 = uniform_filter1d((x * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
108         x_std = np.sqrt(np.maximum(x2 - xm ** 2, 0.0)) / fw
109         ym = uniform_filter1d(y * vis, size, mode="constant") / np.maximum(n, 1e-9)
110         y2 = uniform_filter1d((y * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
111         y_std = np.sqrt(np.maximum(y2 - ym ** 2, 0.0)) / fw
112
113         idx = np.arange(0, N, max(1, int(stride_sec * fps)))
114         X = np.stack([n[idx], spd_mean[idx], spd_max[idx], cross_rate[idx], x_std[idx],
y_std[idx]], axis=1)
115         return np.nan_to_num(X), idx / fps
116

```

43. assistant (2026-06-16T03:13:14.751Z)

Perfect! Now I have all the information I need. Let me create a comprehensive markdown report:

44. user (2026-06-16T03:13:15.572Z)

Chapter marked: "Windowing & signal mapping complete". Continue your current work.

45. assistant (2026-06-16T03:13:57.861Z)

Trajectory-to-Action-Windows: Over-Merge Mechanism & Available Signals

PRIMARY: Windowing Algorithm

```

#### `trajectory_to_action_windows()` ? Core Implementation
**Location:** `backend/pipeline/detectors/tracknet_runner.py:256-323`
**Reused by:** `backend/pipeline/detectors/wasb_runner.py:199` (staticmethod reference)
**Invoked by:** `backend/pipeline/segmenters/trajectory_hybrid.py:197-200`

```

```

python
def trajectory_to_action_windows(
    points: List[TrajectoryPoint],
    fps: float,
    frame_width: float,
    chunk_start: float = 0.0,
    velocity_thresh: float = 0.02,
    merge_gap: float = 2.0,
    min_window_duration: float = 2.0,
    chunk_index: Optional[int] = None,
) -> List[Dict[str, Any]]

```

How "Active" Frames Are Determined

Lines 280-294: Frame is marked "active" when:

1. Shuttle is **visible** (`p.visible == True`)
2. **Inter-frame displacement** (Euclidean distance) normalized by `frame_width` and scaled per-second **exceeds** `velocity_thresh`

```
python
```

Line 290-292: Velocity computation (critical)

```

dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5 # pixels
velocity = (dist / frame_width) * (fps / dframes)
if velocity > velocity_thresh:
    active.append((p.frame, velocity))

```

Velocity Semantics:

- **Per-frame pixel displacement** normalized by `frame_width` (unitless fraction)
- **Multiplied by `fps / dframes`** ? converts to **per-second** scale
- `dframes = p.frame - prev.frame` (can be > 1 if points are sparse)
- **No FPS-aware compensation beyond the `fps / dframes` factor** ? the problem lies here

Active-Region Merging (Dead-Time Bridge)

Lines 299-321: Contiguous groups of active frames separated by ? `merge_gap` seconds are merged into one window:

```
python
```

```

max_gap_frames = int(merge_gap * fps) # Line 299: frame count corresponding to merge_gap
seconds

```

Line 316: if `af[0] - group[-1][0] <= max_gap_frames`: merge

The Problem ? Why It Over-Merges at 59.94fps:

- At **30fps**: `max_gap_frames = int(2.0 * 30) = 60 frames` ? 2.0 sec of silence
- At **59.94fps**: `max_gap_frames = int(2.0 * 59.94) = 119 frames` ? 2.0 sec of silence

The **frame count threshold doubles**, so at higher fps the temporal tolerance is the **same in seconds** but bridges **twice as many frames**. This causes rallies to merge that should be separate if the dead time happens to fall in the 60-119 frame range.

Window Filtering

Line 323: Drops windows shorter than `min_window_duration`:

```
python
```

```

return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

```

Candidate-Window Dict Shape

Lines 301-311 (`_finalize` closure):

```
python
```

```

{
    "start": float,                # seconds; chunk-relative
    "end": float,                  # seconds; chunk-relative

```

```

    "active_frame_count": int,          # count of frames exceeding velocity_thresh
    "peak_velocity": float,            # max velocity in window
    "mean_velocity": float,            # average velocity in window
    "track_id_count": int,             # always 1 (single shuttle)
    "chunk_index": Optional[int],      # source chunk ID for traceability
}
...

```

Configuration & Parameter Sources

```

#### Windowing Parameters & Defaults
| Parameter | Default | Config Key | Source |
|-----|-----|-----|-----|
| `velocity_thresh` | 0.02 | `indexing.<family>.velocity_threshold` | `trajectory_hybrid.py:160`
|
| `merge_gap` | 2.0 sec | `indexing.dead_time_merge_gap` | `trajectory_hybrid.py:161` |
| `min_window_duration` | 2.0 sec | `indexing.min_action_window_duration` |
`trajectory_hybrid.py:162` |

```

****Config Schema Location:**** `backend/config/models.py:117?154`

```

- **TrackNetConfig** (lines 33?51): `velocity_threshold: float = 0.02`
- **WasbIndexConfig** (lines 54?81): `velocity_threshold: float = 0.02`
- **FusionConfig** (lines 84?114): `velocity_threshold: float = 0.02` (family-specific override)
- **IndexingConfig** (lines 117?154):
  - `dead_time_merge_gap: float = 2.0`
  - `min_action_window_duration: float = 2.0`

```

NO Maximum Window Duration or Inactivity-Based Split

****Confirmed Absence:**** The algorithm has ****no upper bound on window duration**** and ****no inactivity timeout that splits long windows****. Once two active regions are merged, the resulting window extends from the first active frame to the last, with no intermediate breaks. This is the second half of the over-merge problem: a long dead-time bridge cements otherwise-separable rallies into a single candidate.

SECONDARY: Available Signals for Better Segmentation

1. Rally Gate (Gate A) ? Net-Crossing Count

****Location:**** `backend/pipeline/detectors/rally\_gate.py:18?117`

****What it computes:****

- Per-window ****shuttle net-crossing count**** ? number of sign changes in (shuttle_coord - net_position)
- Discriminator: real rallies cross the net ?2x; single service feeds cross ~1x
- ****Camera-agnostic****: auto-estimates play axis (x or y based on spread) and net position (median coord)

****Per-frame vs per-window:**** Per-window aggregation of frame-level crossings

****Default on/off:**** ****OFF**** (persisted but not gated;

`FusionSegmenter.\_enrich\_candidate\_stats:80?85`)

****Persistence:**** Stored in `candidate\_segments.stats["net\_crossings"]` as float

****Gating hook:**** `FusionSegmenter.\_select\_for\_handoff:154` ? drops windows with `net\_crossings < min\_crossings` (default 0 = OFF)

****Key API:****

```

...python
gate_windows(points, windows: List[Interval], fps,
              min_crossings=2, deadband_frac=0.06, estimate=None)
-> List[GatedWindow]
...

```

Returns `GatedWindow(start, end, crossings, visible\_points, kept)` per input window.

2. Person Detector ? Motion + Occupancy Cues

```

**Location:** `backend/pipeline/detectors/person_detector.py:1?299` +
`backend/pipeline/detectors/fusion_features.py:1?215`

**What it computes (5 features):**
1. **`players_in_court`** (line `fusion_features.py:97?99`): median per-frame in-court person
count
2. **`two_sided_motion`** (line `fusion_features.py:130?155`): fraction of frames with persons
on both net sides
3. **`mean_player_motion`** (line `fusion_features.py:109?127`): mean per-track centre
displacement (normalised by frame dims)
4. **`in_court_ratio`** (line `fusion_features.py:102?106`): fraction of frames with
?min_players in court
5. **`shuttle_player_colocation`** (line `fusion_features.py:168?189`): fraction of visible-
shuttle frames where shuttle sits inside/adjacent to a person box (held vs in-flight)

**Per-frame vs per-window:** Per-frame detections ? per-window aggregation
**Default on/off:** **OFF** ? person cue only computes if `indexing.fusion.cue_flags["person"]
== True`
**Persistence:** Stored in `candidate_segments.stats` with keys matching `PERSON_FEATURE_NAMES`
(line `fusion_features.py:194?197`)
**Lazy loading:** PersonDetector is instantiated only when person cue is enabled (line
`fusion_hybrid.py:119?120`)

**Key APIs:**
```python

```

## Per-chunk feature extraction (yolo\_hybrid path):

```

extract_action_windows_from_chunk(chunk_path, chunk_start, velocity_thresh, merge_gap)
 -> windows with embedded motion stats

```

## Per-window enrichment (fusion path):

```

detections_per_frame(video_path, start_frame, end_frame, frame_skip)
 -> {"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width", "frame_height"}

compute_person_features(per_frame, shuttle_points, frame_width, frame_height,
 court_polygon=None, net=None, min_players=2)
 -> {feature_name: float} for the 5 features above
...

```

### #### 3. Audio Hit Detection ? Racket-Shuttle Contact Events

```

Location: `backend/pipeline/detectors/audio_features.py:1?34`

What it computes (3 features):
1. **`audio_hit_count`**: number of hit events in window
2. **`audio_hit_rate`**: hits per second
3. **`audio_hit_strength_mean`**: mean strength of detected hits

Per-frame vs per-window: Per-window aggregation of hit events (each has `.t` and
`.strength`)
Default on/off: **OFF** ? expects upstream `HitEvent` list from
`backend/pipeline/audio/hit_detector.py`
Persistence: Columns in `candidate_segments.stats` (line `audio_features.py:15`)

Key API:
```python
compute_audio_features(hits: List[HitEvent], window_start: float, window_end: float)
    -> {"audio_hit_count", "audio_hit_rate", "audio_hit_strength_mean"}
...

```

****Note:**** Hit detection is not integrated into the live pipeline yet (landing note in
`fusion\_hybrid.py:92?98`); this is the schema and compute function only.

4. Fusion Features ? Learned Multi-Signal Combination

```

**Location:** `backend/pipeline/detectors/fusion_features.py:194?215`

```

****What it computes:**** The 5 person features (see above) via `compute_person_features()`
****Per-frame vs per-window:**** Per-window (computed once per candidate window)
****Default on/off:**** ****OFF**** unless `indexing.fusion.cue_flags["person"] == True`
****Persistence:**** Stored directly in `candidate_segments.stats` alongside motion stats

****Integration point:**** `FusionSegmenter._enrich_candidate_stats:76?90` ? always persists `net_crossings`; conditionally adds person features.

5. Fusion Segmenter Gating Hooks

****Location:**** `backend/pipeline/segmenters/fusion_hybrid.py:76?159`

****Two overridable seams:****

- **`\_enrich\_candidate\_stats(cw, points, fps, frame\_width, video\_path, idx\_cfg)`**** (lines 76?90)
 - Adds `net_crossings` (from `rally_gate`, always)
 - Conditionally adds 5 person features (if `cue_flags["person"]`)
 - Returns dict persisted into `candidate_segments.stats`
- **`\_select\_for\_handoff(windows, window\_stats, idx\_cfg)`**** (lines 141?159)
 - ****Served Gate A****: drops windows with `net_crossings < min_crossings`
 - ****Served Gate B****: drops windows with `players_in_court < min_players`
 - Returns indices of kept windows for AI handoff
 - ****Persisted candidates remain unchanged**** (full superset for offline re-scoring)

Candidate Segments DB Schema

****Location:**** `backend/infrastructure/database.py:97?121` (schema v1), `318?339` (insertion)

````sql`

```

CREATE TABLE candidate_segments (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 video_id TEXT NOT NULL,
 detector TEXT NOT NULL,
 chunk_index INTEGER,
 start_time REAL NOT NULL,
 end_time REAL NOT NULL,
 duration REAL NOT NULL,
 confidence REAL,
 stats TEXT, -- JSON: motion + net_crossings + person features
 detection_params TEXT, -- JSON: {velocity_thresh, merge_gap,
min_action_window_duration, ...}
 confirmed_segment_id INTEGER,
 human_label TEXT,
 notes TEXT,
 created_at TIMESTAMP,
 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
 FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON DELETE SET NULL
)

```

**\*\*`stats` JSON keys (persisted per candidate):\*\***

- Motion (always): `peak_velocity`, `mean_velocity`, `active_frame_count`, `track_id_count`
- Gate A (when fusion): `net_crossings`
- Person features (when fusion + person cue ON): `players_in_court`, `two_sided_motion`, `mean_player_motion`, `in_court_ratio`, `shuttle_player_colocation`
- Audio features (when enabled): `audio_hit_count`, `audio_hit_rate`, `audio_hit_strength_mean`

---

#### Root Cause: Why Over-Merge Worsens at 59.94fps

**\*\*The FPS-Sensitivity Bug (documented in `backend/features/rally_seq.py:1?11`):\*\***

At 59.94fps the **window-merging algorithm is unchanged**, but the **frame count threshold doubles**:

- Velocity computation (line `tracknet_runner.py:291`) multiplies by ``fps / dframes``, so a given physical shuttle speed reads **similar** across framerates IF the points are densely sampled.
- **BUT** the merge threshold ``max_gap_frames = int(merge_gap * fps)`` scales linearly with fps.
- A 2-second dead time is always 2 seconds in wall time, but at 59.94fps that's ~120 frames vs ~60 at 30fps.
- Two separate rallies separated by ~1.5s dead time (67?90 frames) stay separated at 30fps but merge at 59.94fps.

**Secondary over-merge mechanism:**

- High-fps trajectories sample more densely, so small jitter/oscillations at the net appear as more "active frames" within the merge window.
- Without an inactivity-timeout split, this extends window duration unchecked.

**Concrete example from eval code** (``backend/eval/calibrate_local.py``, ``calibrate_wasb.py``, ``eval_partial_labels.py``):

- All default to `--fps 59.94`` in their docstrings
- Calibration grids search ``(velocity_thresh, merge_gap, min_window_duration)`` but

**velocity\_thresh** is the only fps-normalized parameter; **merge\_gap** is always seconds.

---

## Summary Map: Segmentation Signals Ready for Better Design

Signal	Location	Scope	Default	Gating	Persistence
<b>Velocity (heuristic)</b>	<code>`tracknet_runner.py:256?323`</code>	Per-frame ? per-window	0.02	None	
<code>`stats.peak_velocity, mean_velocity`</code>					
<b>Net Crossings (Gate A)</b>	<code>`rally_gate.py:91?110`</code>	Per-frame ? count	0		
<code>`fusion_hybrid:154`</code>	<code>`stats.net_crossings`</code>				
<b>Person Motion (Gate B)</b>	<code>`fusion_features.py:97?189`</code>	Per-frame detections ? 5 features			
OFF	<code>`fusion_hybrid:156`</code>	<code>`stats.{players_in_court, two_sided_motion, ?}`</code>			
<b>Audio Hits</b>	<code>`audio_features.py:18?34`</code>	Per-window	OFF	None	
<code>`stats.{audio_hit_count, ?}`</code>					

**To improve over-merge:** Combine Gate A (net crossings ?2 for real rallies) + an **inactivity-timeout split** (max window duration or silence-based break detection) + optional person occupancy (two-sided motion) as a confidence booster.